
FV31S-RMTDK

Instructions

Ver.6

Olympus Corporation

Revision History

Ver. No.	Date	Reason
1	January 2, 2016	Initial version
2	October 3, 2017	Changed the argument of MATL_ROI_CREATE from acquisition to acquisitionType.
3	May 16, 2019	Add an explanation for changing port number to troubleshooting section.
4	September 17, 2019	Add follows to 3.3.Result Code. OUT_OF_MEMORY INVALID_PARAMETERS NO_FRAME UNSUPPORTED_IMAGE UNSUPPORTED_PARAMETERS FILE_NOT_FOUNDUNSUPPORTED_FILE_VERSION UNSUPPORTED_COMPRESSION
5	November 27, 2020	Replaced with sample code compatible with Python 3.7.
6	July 15 , 2021	Corrected a clerical error in getProtocolProgress in 3.2.4

Table of Contents

1. OUTLINE	5
2. SETUP PROCEDURES	5
2.1. HARDWARE SETUP	5
2.2. SOFTWARE SETUP	5
2.2.1. <i>Confirming the connection</i>	5
2.2.2. <i>Installing the development software</i>	6
3. I/F LIST	6
3.1. OUTLINE	6
3.2. I/F SPECIFICATIONS.....	7
3.2.1. <i>Maintenance category</i>	7
3.2.2. <i>Remote category</i>	7
3.2.3. <i>Repeat category</i>	7
3.2.4. <i>Protocol category</i>	8
3.2.5. <i>Canvas category</i>	10
3.2.6. <i>Parameter category</i>	12
3.2.7. <i>settingId</i>	13
3.2.8. <i>Processing category</i>	33
3.2.9. <i>processingId</i>	34
3.2.10. <i>IDA category</i>	35
3.3. RESULT CODE	43
3.4. IMAGE DATA	43
3.4.1. <i>Target image</i>	44
3.4.2. <i>Data configuration</i>	44
3.5. STATE TRANSITION	47
3.6. EXECUTABILITY OF I/F BY STATE.....	48
4. HOW TO USE I/F	54
4.1. PROCEDURES TO CONNECT I/F	55
4.2. PROCEDURES TO SET ARGUMENTS.....	56
4.2.1. <i>Without arguments</i>	56
4.2.2. <i>With arguments</i>	57
4.3. PROCEDURES TO REFER TO RETURN VALUES	59
5. APPLICATION SAMPLES (EXPLANATION OF SAMPLE CODES)	62
5.1. PROCEDURES TO ACCESS TO IMAGE DATA.....	62
5.1.1. <i>Access to File</i>	62
5.1.2. <i>Access to Group</i>	62
5.1.3. <i>Access to Level</i>	63

5.1.4.	<i>Access to Area</i>	63
5.1.5.	<i>Access to Channel</i>	63
5.1.6.	<i>Access to Lambda/Z/T-axis</i>	64
5.1.7.	<i>Access to Image</i>	64
5.1.8.	<i>Acquiring intensity data</i>	66
5.1.9.	<i>Converting to intensity data array</i>	66
5.1.10.	<i>Acquiring image size</i>	66
5.1.11.	<i>Cleaning up</i>	67
5.1.12.	<i>Others</i>	67
5.2.	CELL TRACKING OUT	67
5.2.1.	<i>Sample program</i>	67
5.2.2.	<i>Contents of use case</i>	67
5.2.3.	<i>I/F to be used and order</i>	68
5.2.4.	<i>GUI sample program</i>	73
5.3.	AUTOMATED LARGE FOV MATL SET UP	74
5.3.1.	<i>Sample program</i>	74
5.3.2.	<i>Contents of use case</i>	75
5.3.3.	<i>I/F to be used and order</i>	75
5.4.	LIVE STREAMING	86
5.4.1.	<i>Contents of use case</i>	86
5.4.2.	<i>I/F to be used and order</i>	86
6.	TROUBLESHOOTING	88
7.	TRADEMARK	89

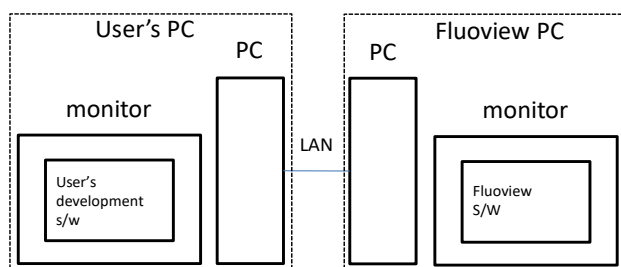
1. Outline

This function (FV31S-RMTDK (hereinafter called “RDK”)) controls the FLUOVIEW system from the software created by the RDK user (hereinafter called “user”) by using various programming languages (Python, Java, C/C++, C#, MATLAB®, LabVIEW, etc.). This document describes the setup procedures to operate RDK, I/F specifications and operating procedures in detail.

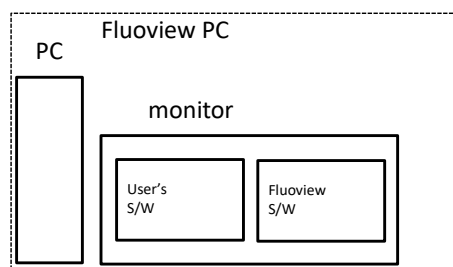
2. Setup procedures

The created software is installed on the PC prepared by the user (see the upper picture) or on the PC where the FLUOVIEW software is installed (see the lower picture).

When using PC prepared by User



When using Fluoview PC



2.1. Hardware setup

When using the PC prepared by the user, two PCs must be connected to the network to execute the HTTP communication between two PCs.

2.2. Software setup

2.2.1. Confirming the connection

When using User's PC

In order to access to the FLUOVIEW software from the software created by the user, the IP Address of the FLUOVIEW PC is necessary. As shown below, execute “ipconfig” on the command prompt of Windows and

record the “IPv4” address.

< Execute on the command prompt of FLUOVIEW PC.>

ipconfig

When the “IPv4” address is acquired, execute the following command from the user’s PC. If the FLUOVIEW PC responds, two PCs are connected.

< Execute on the command prompt of User’s PC. >

ping “IPv4” address acquired

When using FLUOVIEW PC

When FLUOVIEW PC is connected to the network, acquire the “IPv4” address in the same procedures as described above. If it is not connected to the network, the address is 127.0.0.1.

2.2.2. Installing the development software

“RDK” adopts the technology called XMLRPC. In order to use XMLRPC from various programming languages, the libraries for XMLRPC must be prepared. This section introduces the libraries for various languages of which actions have been checked by Olympus. For procedures to install libraries, refer to each library.

Programming language	XMLRPC library
Python	As the XMLRPC library is included, it is not necessary to install the additional library.
Java	Apache XMLRPC library https://ws.apache.org/xmlrpc/
C/C++	XML-RPC for c and c++ http://xmlrpc-c.sourceforge.net/
C#	xml-rpc.net http://xml-rpc.net/
MATLAB	MATLAB can use the Java library. See the section of Java.
LabVIEW	It seems that LabVIEW does not have the library which directly supports XMLRPC. Python can be executed from LabView using procedures described in the following URL: http://www.ni.com/tutorial/8493/en/ The sample codes of LabView in subsequent sections are explained based on this procedure.

3. I/F list

3.1. Outline

This section explains I/F of RDK, definition of specifications, executability in various states and configuration of image data required when using I/F.

3.2. I/F Specifications

3.2.1. Maintenance category

getVersion

Purpose	Get the product version.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	string	version	Version Example: 1.1.1.1
Notes				

3.2.2. Remote category

setRemoteMode

Purpose	Select Remote mode or Local mode.			
Input Parameter	p1	string	mode	Specify LOCAL or REMOTE. If REMOTE mode is specified, the popup dialog is not displayed. If LOCAL mode is specified, the behaviors same as those when operating the software on GUI are executed.
Return Value	p1	string	result	See Result Code.
Notes				

3.2.3. Repeat category

start

Purpose	Start the repeat scan.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
Notes				

stop

Purpose	Stop the repeat scan.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
Notes				

3.2.4. Protocol category

startProtocol

Purpose	Start imaging.			
Input Parameter	p1	string	executionType	EXECUTION_TYPE_MANUAL_MAIN EXECUTION_TYPE_MATL EXECUTION_TYPE_MANUAL_MAIN: Normal imaging (XY, XYT, XYZ, etc.) EXECUTION_TYPE_MATL: MATL
Return Value	p1	string	result	See Result Code.
	p2	integer	wait	Planned time to complete imaging [sec]
	p3	struct	targetName	
	p3-1	string	name	Path of the acquired image EXECUTION_TYPE_MATL: Path of the “omp2info” file
Notes				

stopProtocol

Purpose	Stop imaging.			
Input Parameter	p1	string	executionType	EXECUTION_TYPE_MANUAL_MAIN EXECUTION_TYPE_MATL EXECUTION_TYPE_MANUAL_MAIN: Normal imaging (XY, XYT, XYZ, etc.) EXECUTION_TYPE_MATL:MATL
Return Value	p1	string	result	See Result Code.
Notes				

getProtocolProgress

Purpose	Get progress of imaging.			
Input Parameter	p1	string	executionType	EXECUTION_TYPE_MANUAL_MAIN EXECUTION_TYPE_SM EXECUTION_TYPE_MATL EXECUTION_TYPE_MANUAL_MAIN: Normal imaging (XY, XYT, XYZ, etc.) EXECUTION_TYPE_SM:Sequence Manager EXECUTION_TYPE_MATL: MATL
Return Value	p1	string	result	See Result Code.

	p2	string	state	IDLING SCANNING RESTING WAITING IDLING: State when not imaging SCANNING: State when imaging RESTING: State when not observing during Time-Lapse observation WAITING: Pause state
	p3	struct	cycle	Shows the completed state of cycle. EXECUTION_TYPE_SM Exists only for EXECUTION_TYPE_MATL
	p3-1	integer	total	Planned number of cycles
	p3-2	integer	complete	Number of completed times
	p4	struct	group	Shows the completed state of group. Exists only for EXECUTION_TYPE_MATL
	p4-1	integer	total	Number of planned groups
	p4-2	integer	complete	Number of completed times
	p5	struct	series	Shows the completed state of L-, Z- and T-axis.
	p5-1	integer	totalZ	Number of planned Z
	p5-2	integer	completeZ	Number of completed Z
	p5-3	integer	totalT	Number of planned T
	p5-4	integer	completeT	Number of completed T
	p5-5	integer	totalL	Number of planned L
	p5-6	integer	completeL	Number of completed L
Notes				

getAcquiringFiles

Purpose	Get the file path during acquiring the image.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	struct	targetName	
	p2-1	string	name	File path of the image being acquired.

				EXECUTION_TYPE_MATL: File path of "omp2info"
Notes				

3.2.5. Canvas category

getCanvasIdList

Purpose	Get ID for acquiring the image displayed in the Live image display area.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of string	idList	List of IDs Example: Two IDs in case of 2Ch
Notes				

getSupportedImageFileFormat

Purpose	Get the list of formats when acquiring the image displayed in the Live image display area.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of string	formatList	List of image formats BMP: Bitmap format PNG: PNG format JPG: Jpeg format
Notes				

getSupportedCompressType

Purpose	Get the list of compression types when acquiring the image displayed in the Live image display area.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of string	compressionTypeListList	List of compression types NONE: Not compressed
Notes				

getSupportedColorType

Purpose	Get the list of color types when acquiring the image displayed in the Live image display area.			
Input	p1			

Parameter				
Return Value	p1	string	result	See Result Code.
	p2	array of string	colorType	List of color types GRAYSCALE: Grayscale COLOR: Color image (LUT is loaded)
Notes				

getCanvasImage

Purpose	Get the image displayed in the Live image display area.			
Input Parameter	p1	string	sessionId	Specify UUID. Example: 845d264c-51a9-4811-aaa9-2a2abb09cf27 See Notes for # UUID.
	p2	array of string	canvasIdList	Specify Id corresponding to the channel to be acquired. Id: Specify the string acquired by "getCanvasIdList". To acquire multiple channels, specify multiple Ids in array format.
	p3	string	format	Specify the format of the image data. Format: Specify the string acquired by "getSupportedImageFileFormat".
	p4	string	compress	Specify the compression type of the image data. Type: Specify the string acquired by "getSupportedCompressType".
	p5	string	colorType	Specify the color type of the image data. Type: Specify the string acquired by "getSupportedColorType".
	p6	integer	bitDepth	Always specify 8.
	p7	integer	width	Image size in x direction
	p8	integer	height	Image size in y direction See Notes for # width or height.
Return Value	p1	string	result	See Result Code.
	p2	array of struct	image	
	p2-1	integer	width	Size of the acquired image in x direction
	p2-2	integer	height	Size of the acquired image in y direction
		base64	data	Image data of the acquired image
		bool	continue	Whether there is any remaining image

				data or not True: Yes False: No In case of “True”, the remaining data can be acquired by calling this function.
Notes	<UUID> “UUID” which must be specified by “sessionId” can be created by the standard programming language. For example, following commands can be created by Python. <pre>import uuid uuid.uuid1()</pre> <“height” and “width”> Specify the size of the acquired image. Size of the specified image > Size of the Live display area on GUI ➔ The image of the size of the Live display area on GUI is acquired. Specified image size <= Size of the Live display area on GUI ➔ The image of the size of the specified image is acquired.			

3.2.6. Parameter category

loadParameter

Purpose	Load the observation conditions from the image.			
Input Parameter	p1	string	targetName	File path to the target image for loading the conditions
Return Value	p1	string	result	See Result Code.
Notes				

setParameter

Purpose	Set the parameters.			
Input Parameter	p1	string	settingId	
	p2			Changes depending on “settingId”
Return Value	p1	string	result	See Result Code.
	p2			Changes depending on “settingId”
Notes	For details of “settingId”, see “set” of “settingId”.			

getParameter

Purpose	Get the parameters.			
Input	p1	string	settingId	

Parameter				
	p2			Changes depending on “settingId”
Return Value	p1	string	result	See Result Code.
	p2			Changes depending on “settingId”
Notes	For details of “settingId”, see “set” of “settingId”.			

3.2.7. settingId

LSM_CHANNEL_LIST_SETTING

set

None

get

Purpose	Get the channel list.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of string	channelList	List of channel Ids
Notes				

LSM_CHANNEL_ENABLE_SETTING

set

Purpose	Set Enable/Disable of Channel			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of Channel			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	boolean	enable	True: Enable False: Disable

Notes	
-------	--

LSM_CHANNEL_HV_SETTING

set

Purpose	Get HV of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	integer	hv	Set the “hv” value.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get HV of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	integer	hv	“hv” of the specified channel
Notes				

LSM_CHANNEL_GAIN_SETTING

set

Purpose	Set Gain of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	double	gain	Specify the “gain” value.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Gain of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	double	gain	“gain” of the specified channel
Notes				

LSM_CHANNEL_OFFSET_SETTING

set

Purpose	Set Offset of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	integer	offset	Specify the “offset” value.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Offset of Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	integer	offset	“offset” of the specified channel
Notes				

LSM_LASER_LIST_SETTING

set

None

get

Purpose	Get the list of Lasers.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of struct	laserList	
	p2-1	string	imagingDeviceMode	Specify IMAGING.
	p2-2	string	laserScannerMode	Specify MAIN.
	p2-3	string	phaseId	Phase ID
	p2-4	string	laserType	Laser type SPE or MPE SPE: Single-photon laser MPE: Multi-photon laser
	p2-5	integer	laserWaveLength	Laser wavelength
	p2-6	string	laserId	laserId
Notes				

LSM_CHANNEL_LASER_SETTING

set

Purpose	Set the Laser to Channel.			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	array of struct	laser	
	p2-1	string	imagingDeviceMode	Specify IMAGING.
	p2-2	string	laserScannerMode	Specify MAIN.
	p2-3	string	laserId	Specify the laserId acquired by LSM_LASER_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the information of Laser set to Channel			
Input Parameter	p1	string	channelId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING
Return Value	p1	string	result	See Result Code.
	p2	array of struct		
	p2-1	string	imagingDeviceMode	IMAGING is always set.
	p2-2	string	laserScannerMode	MAIN is always set.
	p2-3	string	laserId	laserId The detail information of the laser can be acquired from the information acquired by this laserId and LSM_LASER_LIST_SETTING.
Notes				

LSM_LASER_ENABLE_SETTING

set

Purpose	Set Enable/Disable of Laser			
Input Parameter	p1	string	laserId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	bool	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of Laser			
Input Parameter	p1	string	laserId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	bool	enable	True: Enable False: Disable
Notes				

LSM_LASER_INTENSITY_SETTING**set**

Purpose	Set the Laser power.			
Input Parameter	p1	string	laserId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
	p2	double	laserIntensity	Laser power
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the Laser power.			
Input Parameter	p1	string	laserId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	double	laserIntensity	Laser power
Notes				

LSM_MICROSCOPE_OBJECTIVE_LENS_LIST_SETTING**set**

None

get

Purpose	Get the list of objectives.			
Input Parameter	p1	string	laserId	Set the channelId acquired by LSM_CHANNEL_LIST_SETTING.
Return Value	p1	string	result	See Result Code.
	p2	array of struct	revolvers	

	p2-1	integer	revolverIndex	Index on the nosepiece (Starts from 0)
	p2-2	string	lensName	Lens name
Notes				

LSM_MICROSCOPE_OBJECTIVE_LENS_CURRENT_SETTING

set

Purpose	Change the nosepiece position of the objective.			
Input Parameter	p1	string	revolverIndex	Specify the Index of the nosepiece to be changed.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the nosepiece position of the objective.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	struct		
	p2-1	integer	revolverIndex	Index on the nosepiece (Starts from 0)
	p2-2	string	lensName	Lens name
Notes				

MATL_ROI_CREATE

set

Purpose	Register MATL ROI.			
Input Parameter	p1	string	acquisitionType	Specify LSM_IMAGING.
	p2	string	roiType	Specify DEFINE_MATRIX.
	p3	integer	column	Field number in X direction
	p4	integer	row	Field number in Y direction
Return Value	p1	string	result	See Result Code.
	p2	string	groupId	ID of the registered MATL Group
Notes				

get

None

MATL_ROI_WELL_DUPLICATE**set**

Purpose	Duplicate MATL ROI.			
Input Parameter	p1	string	srcWellId	Specify the well position where the copy source of MATL ROI exists. Use the alphabet for Y direction and 2-digit number for X direction, such as A01 or C03. If the well is not used, specify A01.
	p2	string	dstWellId	Specify the well position of the copy destination.
Return Value	p1	string	result	See Result Code.
Notes				

get

None

MATL_ROI_GET_REGISTERD_LIST**set**

None

get

Purpose	Get the list of MATL ROI.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	array of struct	groupList	
	p2-1	string	groupId	groupId
	p2-2	string	wellId	Id of the well
Notes				

MATL_ROI_DELETE**set**

Purpose	Delete MATL ROI.			
Input Parameter	p1	string	groupId	groupId of MATL ROI to be deleted
Return Value	p1	string	result	See Result Code.

Notes	
-------	--

get

None

MATL_ROI_ENABLE_SETTING

set

Purpose	Set Enable/Disable of MATL ROI			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of MATL ROI			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
Return Value	p1	string	result	See Result Code.
	p2	boolean	enable	True: Enable False: Disable
Notes				

MATL_ROI_XY_POSITION_SETTING

set

Purpose	Set XY coordinates of MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	x	X coordinates of stage [nm]
	p3	integer	y	Y coordinates of stage [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get XY coordinates of MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI

Return Value	p1	string	result	See Result Code.
	p2	integer	x	X coordinates of stage [nm]
	p3	integer	y	Y coordinates of stage [nm]
Notes				

MATL_ROI_Z_POSITION_SETTING

set

Purpose	Set Z coordinates of MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	z	Z coordinates of stage [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

None

MATL_ROI_Z_ORIGIN_SETTING

set

Purpose	Set Z origin coordinates of MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	zOrigin	Z origin coordinates [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Z origin coordinates of MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
Return Value	p1	string	result	See Result Code.
	p2	integer	zOrigin	Z origin coordinates [nm]
Notes				

MATL_ROI_AREA_DELETE_SETTING

set

Purpose	Delete the specified Area in MATL ROI.			
---------	--	--	--	--

Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	column	Position of the target Area in MATLROI in X direction (Starts from 0.)
	p2	integer	row	Position of the target Area in MATLROI in Y direction (Starts from 0.)
Return Value	p1	string	result	See Result Code.
Notes				

get

None

MATL_ROI_AREA_ENABLE_SETTING

set

Purpose	Set Enable/Disable of the specified Area in MATL ROI			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	column	Position of the target Area in MATLROI in X direction (Starts from 0.)
	p2	integer	row	Position of the target Area in MATLROI in Y direction (Starts from 0.)
	p3	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of the specified Area in MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	column	Position of the target Area in MATLROI in X direction (Starts from 0.)
	p2	integer	row	Position of the target Area in MATLROI in Y direction (Starts from 0.)
Return Value	p1	string	result	See Result Code.
	p3	boolean	enable	True: Enable False: Disable
Notes	During imaging, the Area can be changed from "Enable" to "Disable" only.			

MATL_ROI_AREA_Z_RANGE_SETTING**set**

Purpose	Set the Z observation range of the specified Area in MATL ROI.			
Input Parameter	p1	string	groupId	groupId of target MATL ROI
	p2	integer	column	Position of the target Area in MATLROI in X direction (Starts from 0.)
	p2	integer	row	Position of the target Area in MATLROI in Y direction (Starts from 0.)
Return Value	p1	string	result	See Result Code.
Notes	The Z observation range in the specified Area can be updated by calling this function while the intended Z observation range is set.			

get

None

LSM_BRIGHTNESS_ADJUSTMENT_CREATE_OR_UPDATE_SETTING**set**

Purpose	Register the information of brightness adjustment.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
Notes	The adjustment information can be registered by calling this function while the intended Z position, laser power, hv, gain and offset values are set in advance.			

get

None

LSM_BRIGHTNESS_ADJUSTMENT_ENABLE_SETTING**set**

Purpose	Set Enable/Disable of the brightness adjustment.			
Input Parameter	p1	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of the brightness adjustment.			
---------	--	--	--	--

Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	boolean	enable	True: Enable False: Disable
Notes				

LSM_BRIGHTNESS_ADJUSTMENT_DELETE_ALL_SETTING

set

Purpose	Delete the information of brightness adjustment.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
Notes				

get

None

XY_STAGE_WELL_POSITION_SETTING

set

Purpose	Move the XY stage position to the specified well position.			
Input Parameter	p1	string	wellId	Specify the well position. Use the alphabet for Y direction and 2-digit number for X direction, such as A01 or C03.
	p2	boolean	escapeEnabled	True: The objective escapes while the stage is moving. False: The objective does not escape while the stage is moving.
Return Value	p1	string	result	See Result Code.
Notes				

get

None

XY_STAGE_POSITION_SETTING

set

Purpose	Move the XY stage position.			
Input Parameter	p1	integer	x	X coordinates [nm]
	p2	integer	y	Y coordinates [nm]
	p3	boolean	escapeEnabled	True: The objective escapes while the stage is moving. False: The objective does not escape while the stage is moving.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the XY stage position.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
Input Parameter	p1	integer	x	X coordinates [nm]
	p2	integer	y	Y coordinates [nm]
Notes				

Z_STAGE_POSITION_SETTING

set

Purpose	Move the Z position.			
Input Parameter	p1	integer	stagePosition	Z coordinates (Absolute coordinates) [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the Z position.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	integer	stagePosition	Z coordinates (Absolute coordinates) [nm]
Notes				

LSM_Z_COORDINATE_ENABLE_SETTING

set

Purpose	Set Enable/Disable of Z stack			
Input Parameter	p1	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	boolean	enable	True: Enable False: Disable
Notes				

LSM_Z_COORDINATE_SLICE_NUM_SETTING

set

Purpose	Set the number of slices of Z stack.			
Input Parameter	p1	integer	sliceNum	Number of slices
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the number of slices of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	sliceNum	Number of slices
Notes				

LSM_Z_COORDINATE_STEP_SIZE_SETTING

set

Purpose	Set the step size of Z stack.			
Input Parameter	p1	integer	stepSize	Step size [nm]
Return Value	p1	string	result	See Result Code.

Notes	
-------	--

get

Purpose	Get the step size of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	stepSize	Step size [nm]
Notes				

LSM_Z_COORDINATE_START_SETTING

set

Purpose	Set the start position of Z stack.			
Input Parameter	p1	integer	startPosition	Start position (Relative coordinates) [nm]. Set by the relative coordinates from the Origin position.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the start position of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	startPosition	Start position (Relative coordinates) [nm]
Notes				

LSM_Z_COORDINATE_END_SETTING

set

Purpose	Set the end position of Z stack.			
Input Parameter	p1	integer	endPosition	End position (Relative coordinates) [nm]. Set by the relative coordinates from the Origin position.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the end position of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	endPosition	End position (Relative coordinates) [nm]
Notes				

LSM_Z_COORDINATE_RANGE_SETTING

set

Purpose	Set the range of Z stack.			
Input Parameter	p1	integer	range	Range [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the range of Z stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	range	Range [nm]
Notes				

LSM_Z_COORDINATE_BASE_SETTING

set

Purpose	Set the Z origin position.			
Input Parameter	p1	integer	originPosition	Origin position [nm]
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the Z origin position.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	originPosition	Origin position [nm]

Notes	
-------	--

LSM_Z_COORDINATE_TYPE_SETTING

set

Purpose	Specify the setting methods of the Z observation range and the drive device.			
Input Parameter	p1	string	zType	Z drive device MOTOR/PIEZO MOTOR: Motor PIEZO: Piezo
	p2	string	zType	START_END or RANGE or PIEZO Specify "START_END" or "RANGE" when using the motor as a drive device. Only "START_END" can be specified when using the Piezo as a drive device, START_END: Method to specify the start and end positions. RANGE: Method to specify the range.
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the setting methods of the Z observation range and the drive device.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	string	zType	START_END or RANGE or PIEZO
Notes				

LSM_Z_MOTOR_POSITION_SETTING

set

Purpose	Set the Z motor position.			
Input Parameter	p1	integer	zPosition	Specified Z position [nm]. Relative coordinates
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the Z motor position.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	integer	zPosition	Specified Z positon [nm]. Relative coordinates
Notes				

LSM_Z_PIEZO_POSITION_SETTING

set

Purpose	Set the Piezo position.			
Input Parameter	p1	integer	zPosition	Specified Z positon [nm]. Relative coordinates
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the Piezo position.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	integer	zPosition	Specified Z positon [nm]. Relative coordinates
Notes				

LSM_TIME_LAPS_ENABLE_SETTING

set

Purpose	Set Enable/Disable of T stack.			
Input Parameter	p1	boolean	enable	True: Enable False: Disable
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get Enable/Disable of T stack.			
Input Parameter				

Return Value	p1	string	result	See Result Code.
	p2	boolean	enable	True: Enable False: Disable
Notes				

LSM_TIME_LAPS_REPEAT_TIMES_SETTING

set

Purpose	Set the number of repeat times of T stack.			
Input Parameter	p1	integer	repeatCount	Number of repeat times
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the number of repeat times of T stack.			
Input Parameter				
Return Value	p1	string	result	See Result Code.
	p2	integer	repeatCount	Number of repeat times
Notes				

LSM_TIME_LAPS_INTERVAL_SETTING

set

Purpose	Set the interval of T stack.			
Input Parameter	p1	integer	interval	Interval time: See p2 for the unit.
	p2	integer	unitOfInterval	Specify the unit of the time. $p1 * 10^{-p2}[\text{sec}]$. Example: When $p1=1$ and $p2=0$, the unit is 1[sec]. When $p1=1$ and $p2=1$, the unit is 0.1[sec].
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the interval of T stack.			
Input Parameter				

Return Value	p1	string	result	See Result Code.
	p2	integer	interval	Interval time: See p2 for the unit.
	p3	integer	unitOfInterval	Unit of time. $p1 * 10^{-p2}[\text{sec}]$. Example: When p1=1 and p2=0, the unit is 1[sec]. When p1=1 and p2=1, the unit is 0.1[sec].
Notes				

LSM_SCANNER_TYPE_SETTING

set

Purpose	Set the selection of the scanner.			
Input Parameter	p1	string	scannerType	GALVANO or RESONANT GALVANO: Galvano RESONANT: Resonant
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the scanner.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
	p2	string	scannerType	GALVANO or RESONANT GALVANO: Galvano RESONANT: Resonant
Notes				

LSM_SCANNER_MODE_SETTING

set

Purpose	Set the selection of scanning direction of scanner.			
Input Parameter	p1	string	scannerMode	ONEWAY or ROUNDTRIP ONEWAY: One-way scan ROUNDTRIP: Roundtrip scan
Return Value	p1	string	result	See Result Code.
Notes				

get

Purpose	Get the selection of scanning direction of scanner.			
Input Parameter	p1			
Return Value	p1	string	result	See Result Code.
		string	scannerMode	ONEWAY or ROUNDTRIP ONEWAY: One-way scan ROUNDTRIP: Roundtrip scan
Notes				

3.2.8. Processing category

startProcessing

Purpose	Start processing.			
Input Parameter	p1	string	processingId	ID of the target processing
	p2			Changes depending on the processingId.
Return Value	p1	string	result	See Result Code.
	p2			Changes depending on the processingId.
Notes	For details of processingId, refer to processingId.			

stopProcessing

Purpose	Stop processing.			
Input Parameter	p1	string	processingId	ID of the target processing
	p2			Changes depending on the processingId.
Return Value	p1	string	result	See Result Code.
	p2			Changes depending on the processingId.
Notes	For details of processingId, refer to processingId.			

getProcessingProgress

Purpose	Check the processing progress.			
Input Parameter	p1	string	processingId	ID of the target processing
	p2			Changes depending on the processingId.
Return Value	p1	string	result	See Result Code.
	p2			Changes depending on the processingId.
Notes	For details of processingId, refer to processingId.			

3.2.9. processingId

MATL_STITCH

startProcessing

Purpose	Start the stitching processing.			
Input Parameter	p1	string	targetName	Path to the omp2info file
	p2	string	groupId	GroupID of MATL
	p3	array of struct	stitchSetting	
	p3-1	string	stitchType	Stitching method STITCH_FRAME_WISE STITCH_DEPTH_WISE STITCH_FRAME_WISE: Stitches per Frame which has the same Frame index. STITCH_DEPTH_WISE: Stitches per Frame which has the same Z coordinates. Specify in case when the number of slices in Z stack differs by Area, such as the use case “Automated large FOV MATL set up”.
	p3-2	string	smoothingType	Averaging method NONE GRAVITY_AVERAGE NONE: The stitching boundary is not averaged. GRAVITY_AVERAGE: The stitching boundary is averaged.
	p3-3	integer	zMatchIndex	Z index of the Frame to execute the template matching during stitching.
	p3-4	integer	tMatchIndex	T index of the Frame to execute the template matching during stitching.
	p3-5	integer	lMatchIndex	Lambda index of the Frame to execute the template matching during stitching.
	p3-6	integer	scale	Reduction ratio [%] If the scale is not reduced, set to 100. If the scale is reduced to half, set to 50.
	p3-7	array of	matchingChannelIds	ID of Channel used for template matching.

		integer		Specify ID acquired by “getProperty (EnabledChannelIdList)” in IDA category. For example, if using only Ch2 for template matching, specify the second ID in the list acquired by “EnabledChannelIdList” described above.
Return Value	p1	string	result	See Result Code.
	p2	string	stitchFilePath	Path to the image file of the stitching results
Notes				

stopProcessing

Purpose	Stop the stitching processing.			
Input Parameter	p1	string	targetName	Path to the omp2info file
	p2	string	groupId	GroupID of MATL
Return Value	p1	string	result	See Result Code.
Notes				

getProcessingProgress

Purpose	Check the progress of stitching processing.			
Input Parameter	p1	string	targetName	Path to the omp2info file
	p2	string	groupId	GroupID of MATL
Return Value	p1	string	result	See Result Code.
	p2	array of struct	progress	
	p2-1	integer	total	Total number: 100[%] is always set.
	p2-2	complete	complete	Number of completed times [%]
	p3	array of string	stitchFilePaths	Path to the stitched image.
Notes				

3.2.10. IDA category

open

Purpose	Open the specified image.
---------	---------------------------

Input Parameter	p1	string	filename	File path of the image
Return Value	p1	string	result	See Result Code.
	p2	string	fileId	File ID
Notes				

close

Purpose	Close the specified image.			
Input Parameter	p1	string	fileId	File ID Specify ID acquired by I/F of "open".
Return Value	p1	string	result	See Result Code.
Notes				

getNumOfGroups

Purpose	Get the number of groups in the file.			
Input Parameter	p1	string	fileId	File ID Specify ID acquired by I/F of "open".
Return Value	p1	string	result	See Result Code.
	p2	integer	numOfGroups	Number of groups
Notes	If the "oir" file is specified, the value is 1. If the "omp2info" file is specified, the value is larger than 1.			

getGroup

Purpose	Get the group ID.			
Input Parameter	p1	string	fileId	File ID Specify ID acquired by I/F of "open".
	p2	integer	group	Index of the group (Starts from 0) Up to the number of groups acquired by "getNumOfGroups" -1 can be specified.
Return Value	p1	string	result	See Result Code.
	p2	string	groupId	Group ID
Notes				

releaseGroup

Purpose	Release the group ID acquired.			
Input Parameter	p1	string	groupId	Group ID Specify ID acquired by "getGroup".
Return Value	p1	string	result	See Result Code.
Notes				

getNumOfLevel

Purpose	Get the number of levels in the group.			
Input Parameter	p1	string	groupId	Group ID Specify ID acquired by “getGroup”.
Return Value	p1	string	result	See Result Code.
	p2	integer	numOfLayers	Number of levels
Notes	This is an extendable I/F in the future and it always returns 1.			

getLevelImageSize

Purpose	Get the image size of the specified level.			
Input Parameter	p1	string	groupId	Group ID Specify ID acquired by “getGroup”.
	p2	integer	level	Index of the level (Starts from 0) Up to the number acquired by “getNumOfLevel”-1 can be specified.
Return Value	p1	string	result	See Result Code.
	p2	integer	width	X size of the image
	p2-1	integer	height	Y size of the image
Notes				

getNumOfArea

Purpose	Get the number of areas in the specified group.			
Input Parameter	p1	string	groupId	Group ID Specify ID acquired by “getGroup”.
	p2	integer	level	Index of the level (Starts from 0) Up to the number acquired by “getNumOf”-1 can be specified.
Return Value	p1	string	result	See Result Code.
	p2	integer	numOfArea	Number of areas
Notes				

getArea

Purpose	Get the area ID.			
Input Parameter	p1	string	groupId	Group ID Specify ID acquired by “getGroup”.
	p2	integer	level	Index of the level (Starts from 0) Up to the number acquired by “getNumOfLevel” -1 can be specified.
	p3	integer	area	Index of the area (Starts from 0)

				Up to the number acquired by “getNumOfArea”-1 can be specified.
Return Value	p1	string	result	See Result Code.
	p2	string	areald	Area ID
Notes				

releaseArea

Purpose	Release the Area ID acquired.			
Input Parameter	p1	integer	areald	Area ID Specify ID acquired by “getArea”.
Return Value	p1	string	result	See Result Code.
Notes				

getProperty

Purpose	Get the property information of the area.																			
Input Parameter	p1	integer	areald	Area ID Specify ID acquired by “getArea”.																
	p2	array of struct																		
	p2-1	string	key	Property name acquired Settable property names are described below.																
				<table><tr><th>Name</th><th>Meaning</th></tr><tr><td>EnabledChannelIdList</td><td>List of IDs of logical channels in the image.</td></tr><tr><td>ChannelInfo</td><td>Detail information of the channel</td></tr><tr><td>Axes</td><td>Information of the image</td></tr><tr><td>AxisStartPosition</td><td>Start position of the axis</td></tr><tr><td>AxisEndPosition</td><td>End position of the axis</td></tr><tr><td>AxisStep</td><td>Number of steps of the axis</td></tr><tr><td>AxisMaxSize</td><td>Maximum number of axis</td></tr></table>	Name	Meaning	EnabledChannelIdList	List of IDs of logical channels in the image.	ChannelInfo	Detail information of the channel	Axes	Information of the image	AxisStartPosition	Start position of the axis	AxisEndPosition	End position of the axis	AxisStep	Number of steps of the axis	AxisMaxSize	Maximum number of axis
				Name	Meaning															
				EnabledChannelIdList	List of IDs of logical channels in the image.															
				ChannelInfo	Detail information of the channel															
				Axes	Information of the image															
				AxisStartPosition	Start position of the axis															
				AxisEndPosition	End position of the axis															
				AxisStep	Number of steps of the axis															
AxisMaxSize	Maximum number of axis																			
p2-2	struct	additional	Required only when followings are keys. ChannelInfo AxisStartPosition AxisEndPosition AxisStep																	

				AxisMaxSize
	p2-2-1	string	channelId	< When ChannelInfo is a key > Set elements of EnabledChannelIdList.
			axisName	< When others excluding above are keys > Specify TIMELAPSE or ZSTACK or LAMBDA. TIMELAPSE: T-axis information ZSTACK: Z-axis information LAMBDA: λ-axis information
Return Value (In case of EnabledChannelIdList)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of string		List of IDs of channels in the image (separated by comma)
Return Value (In case of ChannelInfo)	p1	string	result	See Result Code.
	p2	array of struct	structValue	
	p2-1	string	deviceName	Device name of the channel
		string	name	Channel name
		integer	bitCount	Effective bit length/pixel [bit]
		string	colorSpace	GRAYSCALE or COLOR
		integer	depth	Bit length /pixel [byte]
		integer	order	Order of channels on the display
Return Value (In case of Axes)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of string		List of axes (separated by comma)
Return Value (In case of AxisStartPosition)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of		Start position

		doulble		
Return Value (In case of AxisEndPosition)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of doulble		End position
Return Value (In case of AxisStepPosition)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of doulble		Step value
Return Value (In case of AxisStepPosition)	p1	string	result	See Result Code.
	p2	array of struct	value	
		array of integer		Maximum value
Notes	"ChannelInfo" cannot be used for the image being acquired.			

Information acquired by ChannelInfo

deviceName	Imaging device of channel
name	Channel

Imaging device name of channel: deviceName

Channel name: name

Effective bit number: bitCount

Number of bytes in 1pixel

Order of channels

getImage

Purpose	Get the image ID.			
Input	p1	string	areald	Area ID

Parameter				Specify ID acquired by “getArea”.
	p2	string	channelId	Specify the channelId acquired by “getProperty/EnabledChannelIdList”.
	p3	array of struct	axisInfo	
	p3-1	string	axisName	Specify TIMELAPSE or ZSTACK or LAMBDA. TIMELAPSE: T-axis information ZSTACK: Z-axis information LAMBDA: λ-axis information
	p3-2	integer	index	Specify the index of the Frame to be acquired of the axis specified by “p3-1”. (Starts from 0)
Return Value	p1	string	result	See Result Code.
	p2	array of string	imageId	Array of imageID
Notes				

releaseImage

Purpose	Release the image ID acquired.			
Input Parameter	p1	string	imageId	Area ID Specify ID acquired by “getImage”.
Return Value	p1	string	result	See Result Code.
Notes				

getImageBody

Purpose	Get the intensity data of the image.			
Input Parameter	p1	string	imageId	Image ID Specify ID acquired by “getImage”.
	p2	string	compressType	Specify NONE.
	p3	integer	x	Specify 0.
	p4	integer	y	Specify 0.
	p5	integer	width	Specify the X size value of the image.
	p6	integer	height	Specify the Y size value of the image.
Return Value	p1	string	result	See Result Code.
	p2	boolean	continue	Whether there is any remaining image data or not. True: Yes False: No

				In case of “True”, the remaining image data can be acquired by calling “getNextImageBody”.
	p3	array of byte	data	Array of intensity data
Notes				

getNextImageBody

Purpose	Get the intensity data of the image.			
Input Parameter	p1	string	imageId	Image ID Specify ID acquired by “getImage”.
Return Value	p1	string	result	See Result Code.
	p2	boolean	continue	Whether there is any remaining image data or not. True: Yes False: No In case of “True”, the remaining image data can be acquired by calling “getNextImageBody”.
	p3	array of byte	data	Array of intensity data
Notes				

getImageSize

Purpose	Get the image size of the image.			
Input Parameter	p1	string	imageId	Image ID Specify ID acquired by “getImage”.
Return Value	p1	string	result	See Result Code.
	p2	integer	width	X size of the image
	p2-1	integer	height	Y size of the image
Notes				

getImageAxis

Purpose	Get the index in the axis of the image.			
Input Parameter	p1	string	imageId	Image ID Specify ID acquired by “getImage”.
Return Value	p1	string	result	See Result Code.
	p2	array of struct	axisInfo	

	p2-1	string	axisName	TIMELAPSE or ZSTACK or LAMBDA TIMELAPSE: T-axis information ZSTACK: Z-axis information LAMBDA: λ -axis information
	p2-2	integer	index	Index of the axis specified by "p2-1" (Starts from 0)
Notes				

getLUT

Purpose	Get the LUT information.			
Input Parameter	p1	integer	areaid	Area ID Specify ID acquired by "getArea".
	p2	string	channelId	Specify the channelId acquired by "getProperty/EnabledChannelIdList".
Return Value	p1	string	result	See Result Code.
	p2	array of byte	rData	LUT table of RED
	p2-1	array of byte	gData	LUT table of GREEN
	p2-2	array of byte	bData	LUT table of BLUE
Notes				

3.3. Result Code

Result Code	Meaning
OK	Success
ERROR	Failure
UNAVAILABLE	Unavailable state
OUT_OF_MEMORY	Insufficient memory
INVALID_PARAMETERS	Invalid parameters
NO_FRAME	The specified image was not found
UNSUPPORTED_IMAGE	Unsupported image
UNSUPPORTED_PARAMETERS	Unsupported parameters
FILE_NOT_FOUND	File not found
UNSUPPORTED_FILE_VERSION	Unsupported file version

3.4. Image Data

When using the I/F in the IDA category, several concepts related to the image data must be understood.

This section describes these concepts.

3.4.1. Target image

The I/F in the IDA category supports three image formats: “oir”, “omp2info” and “vs”.

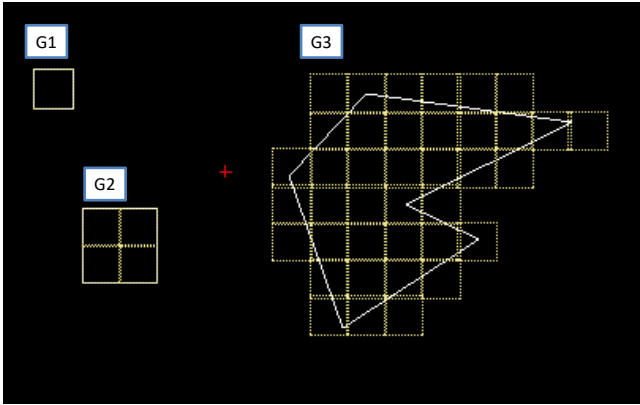
“oir” is an image format normally outputted by this software.

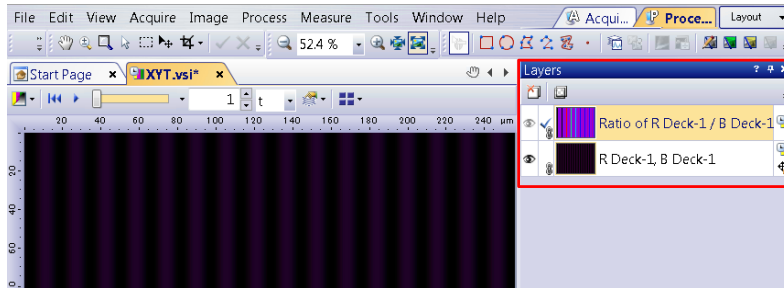
“omp2info” is an image format created during MATL imaging.

“vs” is an image format normally outputted by cellSens software.

3.4.2. Data configuration

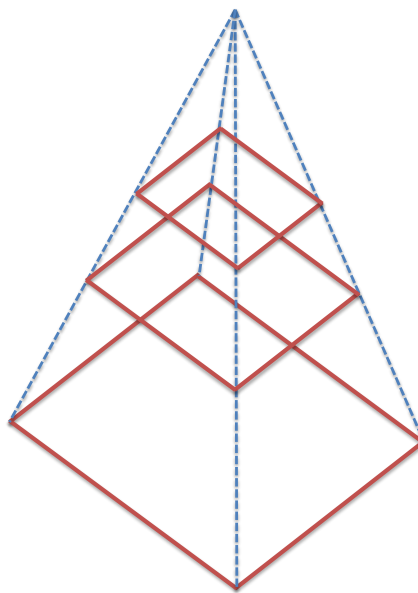
In order to deal with these 3 types of image format in a unified manner, the data configuration composed of Group, Level and Area is used.

Term	Meaning	
Group	“omp2info” and “vsi” formats include multiple image data. “Group” is a frame to deal with such multiple image data.	
	oir	As “oir” does not include multiple image data, the number of Group is 1.
	omop2Info	This format is equivalent to each group when executing MATL. Therefore, the number of Groups is equivalent to the number of groups of MATL. 
	vsi	This format is displayed as a Layer on UI of cellSens software. Therefore, the number of Groups is equivalent to the number of Layers.



Level

The vsi image includes multiple images with different resolutions because the skipped processing is applied to the image data in order to accelerate the drawing processing speed. "Level" is each image with different resolution.
Example: If the image of 2K x 2K and 512 x 512 is created based on the 4K x 4K image, there are three Levels.



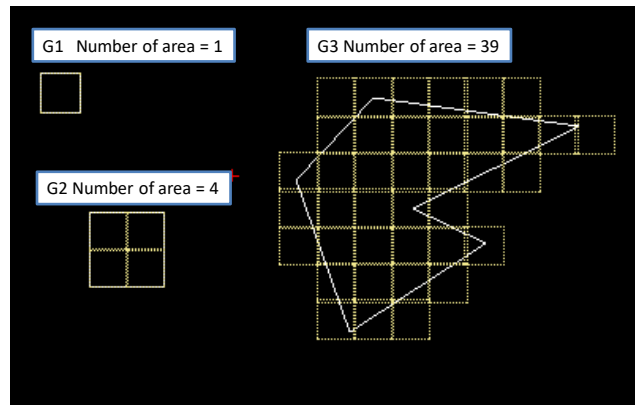
Level2 = 512 x 512

Level1 = 2k x 2k

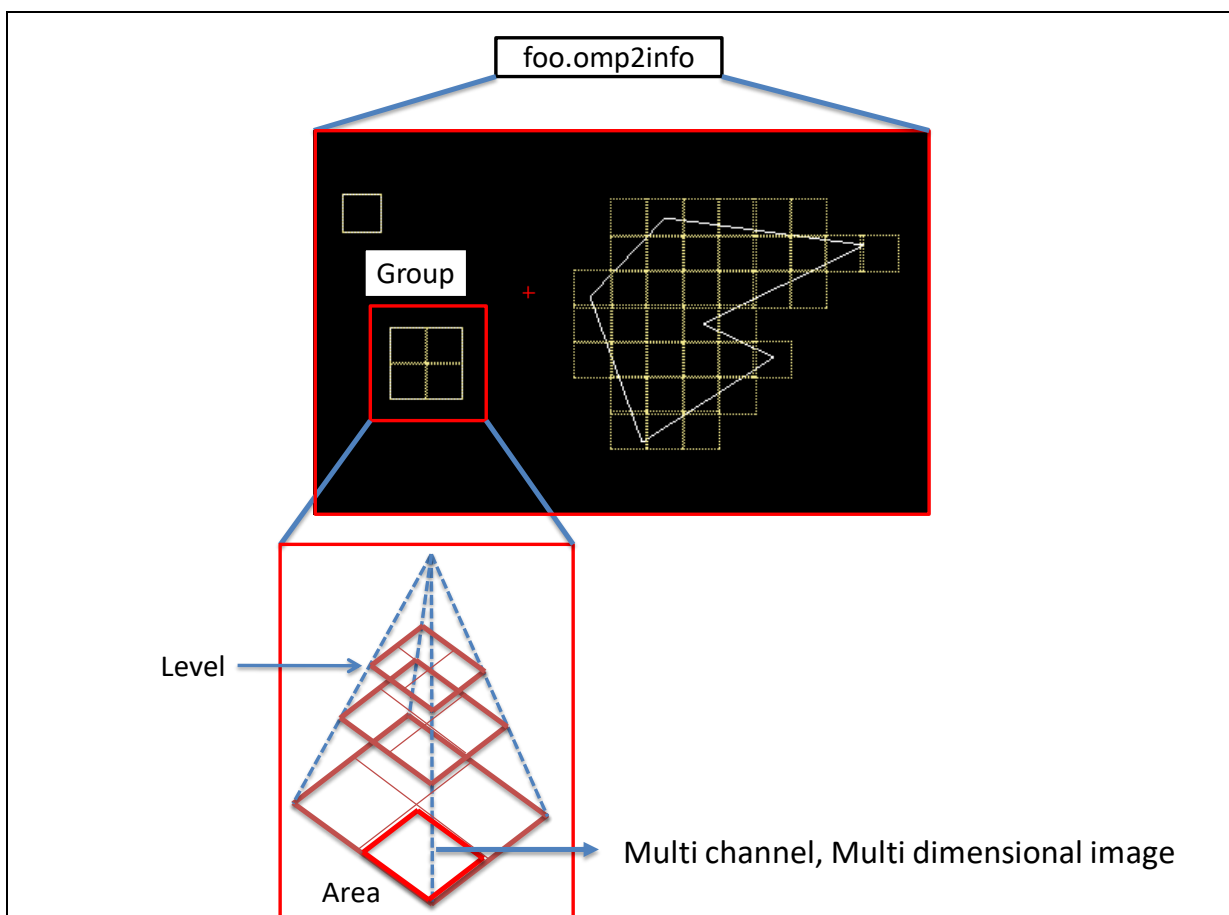
Level0 = 4k x 4k

Area

In some cases, one Group is divided into multiple sections. "Area" is each image divided.
Example: See the following picture.



The following diagram illustrates the Group/Level/Area conceptually.



Namely, one image includes multiple Groups. Each Group includes multiple Levels. Each Level is composed of multiple Areas. Each Area is an image which has multi-channels and multi-dimensions.

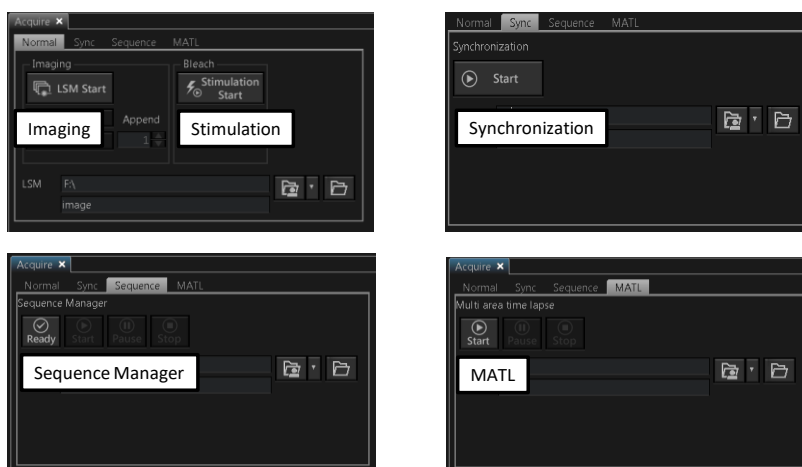
Refer to “Procedures to access to image data” for procedures to access to the particular image data.

3.5. State Transition

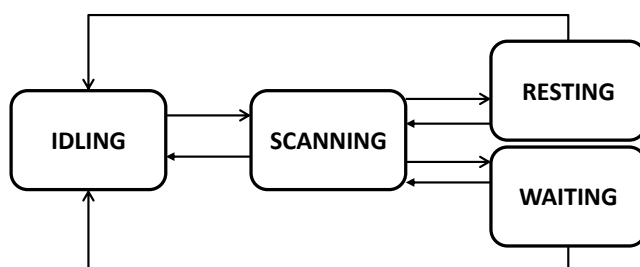
I/F described above can be executed or cannot be executed depending on the internal state of software. Basically, I/F is designed as “If the state is same as the state when using GUI of FLUOVIEW software, corresponding I/F can be used, but if not, corresponding I/F cannot be used.” The subsequent sections define the approximate state to get the quick overview conveniently and describe the executability by state. Therefore, since the state defined in this document is approximate, even though “ok” is described, it could be “ng” depending on the difference in state. In such a case, please operate GUI corresponding to the intended I/F and determine whether they are available or not.

When defining the state, it is important whether the imaging or the stimulation is executed or not. Names of imaging function, stimulation function or other functions combining those are defined as follows.

Function name	Meaning
Imaging	See the upper left picture shown below.
Stimulation	See the upper left picture shown below.
Synchronization	See the upper right picture shown below.
Sequence Manager	See the lower left picture shown below.
MATL	See the lower right picture shown below.



Then, states are defined as follows.



State name	Meaning
IDLING	State where none of Imaging, Stimulation, Synchronization, Sequence Manager and MATL is working
SCANNING	State where either one of Imaging, Stimulation, Synchronization, Sequence Manager or MATL is working
RESTING	State where either one of Imaging, Synchronization, Sequence Manager or MATL is working and state of rest time of Time-Lapse observation
WAITING	State where MATL is working and Pause is executed.

3.6.Executability of I/F by state

This section defines the executability of I/F described above by state.

“ok”: Available; “ng”: Unavailable; n/a: Not supported

I/F	IDLING	SCANNING	RESTING	WAITING
Maintenance				
getVersion	ok	ok	ok	ok
Remote				
setRemoteMode	ok	ok	ok	ok
Repeat				
start	ok	ng	ok	ng
stop	ng	ng	ok During repeat scan only	ok During repeat scan only
Protocol				
startProtocol	ok	ng	ng	ng
stopProtocol	ng	ok	ok	ok
getProtocolProgress	ok State finishe d last	ok	ok	ok
getAcquiringFiles	ok	ok	ok	ok
Canvas				
getCanvasIdList	ok	ok	ok	ok
getSupportedImageFileFormat	ok	ok	ok	ok
getSupportedCompressType	ok	ok	ok	ok
getSupportedColorType	ok	ok	ok	ok
getCanvasImage	ok	ok	ok	ok
Parameter				

	loadParameter	ok	ng	ng	ng
LSM_CHANNEL_LIST_SETTING					
	setParameter	n/a	n/a	n/a	n/a
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_CHANNEL_ENABLE_SETTING					
	setParameter	ok	ng	ng	ng
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_CHANNEL_HV_SETTING					
	setParameter	ok	ok *λ: NG *MATL: NG *V-Ch:NG	ok	n/a
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_CHANNEL_GAIN_SETTING					
	setParameter	ok	ok	ok	n/a
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_CHANNEL_OFFSET_SETTING					
	setParameter	ok	ok	ok	n/a
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_LASER_LIST_SETTING					
	setParameter	n/a	n/a	n/a	n/a
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		
LSM_CHANNEL_LASER_SETTING					
	setParameter	ok	ng	ng	ng
	getParameter	ok	“n/a” while executing Sequence Manager or MATL		

LSM_LASER_ENABLE_SETTING					
	setParameter	ok	ng	ng	ng
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
LSM_LASER_INTENSITY_SETTING					
	setParameter	ok	ok	ok	ok
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
LSM_MICROSCOPE_OBJECTIVE_LENS_LIST_SETTING					
	setParameter	n/a	n/a	n/a	n/a
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
LSM_MICROSCOPE_OBJECTIVE_LENS_CURRENT_SETTING					
	setParameter	ok	ng	ng	ng
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
MATL_ROI_CREATE					
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
MATL_ROI_WELL_DUPLICATE					
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
MATL_ROI_GET_REGISTERD_LIST					
	setParameter	n/a	n/a	n/a	n/a
	getParameter	ok	ok	ok	ok
MATL_ROI_DELETE					
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
MATL_ROI_ENABLE_SETTING					
	setParameter	ok	ng	ok	ok
	getParameter	n/a	n/a	n/a	n/a
MATL_ROI_XY_POSITION_SETTING					
	setParameter	ok	ng	ok	ok
	getParameter	ok	ok	ok	ok
MATL_ROI_Z_POSITION_SETTING					
	setParameter	ok	ng	ok	ok

	getParameter	ok	ok	ok	ok
	MATL_ROI_Z_ORIGIN_SETTING				
	setParameter	ok	ng	ok	ok
	getParameter	ok	ok	ok	ok
	MATL_ROI_AREA_DELETE_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
	MATL_ROI_AREA_ENABLE_SETTING				
	setParameter	ok	ng	ok	ok
	getParameter	ok	ok	ok	ok
	MATL_ROI_AREA_Z_RANGE_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
	LSM_BRIGHTNESS_ADJUSTMENT_CREATE_OR_UPDATE_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
	LSM_BRIGHTNESS_ADJUSTMENT_ENABLE_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
	LSM_BRIGHTNESS_ADJUSTMENT_DELETE_ALL_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
	XY_STAGE_WELL_POSITION_SETTING				
	setParameter	ok	ng	ng	ng
	getParameter	n/a	n/a	n/a	n/a
	XY_STAGE_POSITION_SETTING				
	setParameter	ok	ok MATL: ng	ok	ok
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence Manager or MATL		
	Z_STAGE_POSITION_SETTING				
	setParameter	ok	XYT scan: ok Others: ng	ok	ok
	getParameter	ok	ok	ok	ok
			"n/a" while executing Sequence		

				Manager or MATL		
		LSM_Z_COORDINATE_ENABLE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_SLICE_NUM_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_STEP_SIZE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_START_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_END_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_RANGE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_BASE_SETTING				
		setParameter	ok	ng	ok	ok
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_COORDINATE_TYPE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence		

				Manager or MATL		
		LSM_Z_MOTOR_POSITION_SETTING				
		setParameter	ok	XYT scan: ok Others: ng	ok	ok
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_Z_PIEZO_POSITION_SETTING				
		setParameter	ok	XYT scan: ok Others: ng	ok	ok
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_TIME_LAPS_ENABLE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_TIME_LAPS_REPEAT_TIMES_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_TIME_LAPS_INTERVAL_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_SCANNER_TYPE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok
				"n/a" while executing Sequence Manager or MATL		
		LSM_SCANNER_MODE_SETTING				
		setParameter	ok	ng	ng	ng
		getParameter	ok	ok	ok	ok

				“n/a” while executing Sequence Manager or MATL		
Processing						
	MATL_STITCH "*"ok" for all states excluding the case when acquiring the target image.					
		startProcessing	ok	ok*	ok*	ok*
		stopProcessing	ok	ok*	ok*	ok*
		getProcessingProgress	ok	ok*	ok*	ok*
IDA						
		open	ok	ok	ok	ok
		close	ok	ok	ok	ok
		getNumOfGroups	ok	ok	ok	ok
		getGroup	ok	ok	ok	ok
		releaseGroup	ok	ok	ok	ok
		getNumOf	ok	ok	ok	ok
		getLevellImageSize	ok	ok	ok	ok
		getNumOfArea	ok	ok	ok	ok
		getArea	ok	ok	ok	ok
		releaseArea	ok	ok	ok	ok
		getProperty	ok	ok	ok	ok
		getImage	ok	ok	ok	ok
		releaselImage	ok	ok	ok	ok
		getImageBody	ok	ok	ok	ok
		getNextImageBody	ok	ok	ok	ok
		getImageSize	ok	ok	ok	ok
		getImageAxis	ok	ok	ok	ok
		getLUT	ok	ok	ok	ok

4. How to use I/F

This section explains how to use I/F taking some I/F as examples.

The particular program codes are explained using Python, Java, C/C++, C#, MATLAB and LabVIEW.

Refer to following paths of FV31S-RMTDK for programs described in this section.

FV31S-RMTDK/SampleProgram/

C#/HowToUseRDK/HowToUseRDK.sln

C_C++/HowToUseRDK/HowToUseRDK.sln

Java/HowToUseRDK/src/HowToUseRDK.java

LabView/ExecutePythonScript.vi

Matlab/HowToUseRDK.m

Python/HowToUseRDK.py

4.1. Procedures to connect I/F

The I/F can be connected by setting URL to libraries for programming languages.

“url” is <http://IPAddress of Connected PC:8080/xmlrpc>.

In the followings, the IPAdress 10.100.100.100 changes according to execution environments. For procedures to acquire the IPAdress, see “Confirming the connection”.

<Python>

```
import xmlrpc.client
url = "http://10.100.100.100:8080/xmlrpc"
proxy = xmlrpc.client.ServerProxy(url)
```

<Java>

```
XmlRpcClient proxy = new XmlRpcClient();
XmlRpcClientConfigImpl config = new XmlRpcClientConfigImpl();
URL url = new URL("http://10.100.100.100:8080/xmlrpc");
config.setServerURL(url);
proxy.setConfig(config);
```

<C/C++>

```
std::string const strURL( "http://10.100.100.100:8080/xmlrpc" );
xmlrpc_c::clientSimple Proxy;
xmlrpc_c::value Result, Result2;
```

<C#>

```
public interface IRDK : IXmlRpcProxy
{
    [XmlRpcMethod("Maintenance.getVersion")]
    XmlRpcStruct getVersion();
    [XmlRpcMethod("Protocol.startProtocol")]
    XmlRpcStruct startProtocol(XmlRpcStruct Input);
}
class Program
{
    static void Main(string[] args)
    {
        //connect
        IRDK Proxy = XmlRpcProxyGen.Create<IRDK>();
        Proxy.Url = "http://10.100.100.100:8080/xmlrpc";
    }
}
```

<MATLAB>

```
proxy = javaObject('org.apache.xmlrpc.client.XmlRpcClient')
config = javaObject('org.apache.xmlrpc.client.XmlRpcClientConfigImpl')
url = javaObject('java.net.URL', 'http:// 10.100.100.100:8080/xmlrpc')
config.setServerURL(url)
proxy.setConfig(config)
```

<LabVIEW>

The LabVIEW can be connected via Python. Refer to the sample of Python in Section 4.1.1 Without arguments.

4.2. Procedures to set arguments

4.2.1. Without arguments

Some I/F do not have arguments. In such a case, you can call I/F without inputting arguments as shown below.

<python>

```
result = proxy.Maintenance.getVersion()
```

<Java>

```
Object result = proxy.execute("Maintenance.getVersion",Collections.emptyList());
```

<C/C++>

```
std::string const strGetVersion("Maintenance.getVersion");
Proxy.call(strURL, strGetVersion, "", &Result);
```

<C#>

```
XmlRpcStruct Result1 = Proxy.getVersion();
```

<MATLAB>

```
result1 = proxy.execute('Maintenance.getVersion','');
```

<LabVIEW>

To execute from LabVIEW, execute Python from LabVIEW and Python calls I/F of RDK.
“ExecutePythonScript.vi” is prepared as a sample program. This “vi” has following GUIs.

working directory
C:\LabView

python script name
HowToUseRDK.py

python script arguments
getVersion

standard output

standard error

Set following values to respective variables on GUI.

Working directory: Select the folder which includes Python Script.

Python script name: Set the name of Python Script which is in the working directory.

Python script arguments: Set the argument which will be transferred to Python Script.

“HowToUseRDK.py” is prepared as Python Script to be executed. This sample is composed of following codes and if the name of I/F to be executed as an argument is transferred, that I/F is executed.

To execute I/F of “getVersion”, input “getVersion” to python script arguments and execute it. When “getVersion” is executed, the following return value can be acquired. The return value to be acquired on the LabVIEW side can be changed within Python codes.

standard output

```
{'version': '2.1.1.70', 'result': 'OK'}
```

4.2.2. With arguments

All arguments of RDK are the array type with xmlrpc specifications. Each programming language has the argument type corresponding to the array type. Therefore, RDK arguments can be set by using that type.

For example, when calling “startProtocol”, follow the procedures described below.

This I/F have one string type argument as explained above. To call this I/F, follow the procedures described below.

Input Parameter	p1	string	executionType	EXECUTION_TYPE_MANUAL_MAIN EXECUTION_TYPE_MATL EXECUTION_TYPE_MANUAL_MAIN: Normal imaging (XY, XYT, XYZ, etc.) EXECUTION_TYPE_MATL:MATL
--------------------	----	--------	---------------	---

<Python>

```
inputVal = {"executionType": "EXECUTION_TYPE_MANUAL_MAIN"}
result = proxy.Protocol.startProtocol(inputVal)
```

<Java>

```
Map<String, Object> input = new HashMap<>();
input.put("executionType", "EXECUTION_TYPE_MANUAL_MAIN");
result = proxy.execute("Protocol.startProtocol", Collections.singletonList(input));
```

<C/C++>

```
std::string const strStartProtocol("Protocol.startProtocol");
std::map<std::string, xmlrpc_c::value> Param;
std::pair<std::string, xmlrpc_c::value> ParamMem1("executionType",
xmlrpc_c::value_string("EXECUTION_TYPE_MANUAL_MAIN"));
Param.insert(ParamMem1);
xmlrpc_c::value_struct const paramSt(Param);
xmlrpc_c::paramList params;
params.add(paramSt);
Proxy.call(strURL, strStartProtocol, params, &Result2);
```

<C#>

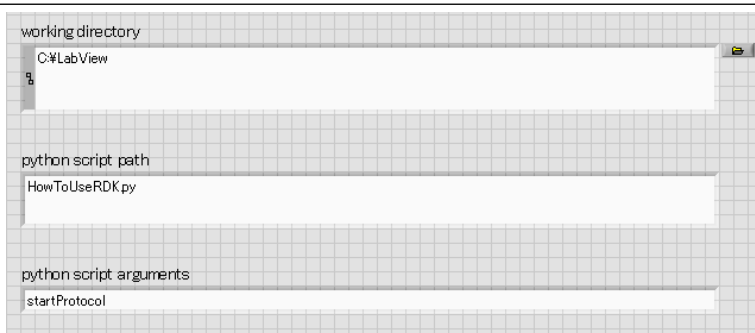
```
XmlRpcStruct Input = new XmlRpcStruct();
Input.Add("executionType", "EXECUTION_TYPE_MANUAL_MAIN");
XmlRpcStruct Result2 = Proxy.startProtocol(Input);
```

<MATLAB>

```
input=java.util.HashMap
input.put('executionType','EXECUTION_TYPE_MANUAL_MAIN')
result2 = proxy.execute('Protocol.startProtocol', input )
```

<LabVIEW>

Execute LabVIEW by inputting startProtocol to python script arguments. Arguments specified by other languages are specified in the python program.



4.3. Procedures to refer to return values

The return value type of I/F of RDK is always the "struct" type. The "struct" always includes the "result" key and the success or failure of I/F are stored. The "struct" may include other keys according to I/F.

This section explains the procedures to work with return values of "startProtocol" described previously as an example.

This function has following three return values as described previously. To refer to the return value of this I/F, follow the procedures shown below.

Return Value	p1	string	result	See Result Code.
	p2	integer	wait	Planned time to complete imaging [sec]
	p3	struct	targetName	
	p3-1	string	name	Path of the acquired image EXECUTION_TYPE_MATL: Path of the omp2info file.

<Python>

```
result = proxy.Protocol.startProtocol(inputVal)
# result
print(result['result'])
# wait
print(result['wait'])
# targetName
print(targetName = result['targetName'])
```

<Java>

```
Map<String, Object> input = new HashMap<>();
input.put("executionType", "EXECUTION_TYPE_MANUAL_MAIN");
result = proxy.execute("Protocol.startProtocol", Collections.singletonList(input));
@SuppressWarnings("unchecked")
Map<String, Object> result2 = (Map<String, Object>) result;
if ( result2.get("result").equals("OK")){
    //wait
    System.out.println(result2.get("wait"));
    //targetName
```

```

        Object[] objTargetNames = (Object[])result2.get("targetName");
        for (Object objTargetName : objTargetNames) {
            @SuppressWarnings("unchecked")
            Map<String, Object> mapTargetName = (Map<String, Object>) objTargetName;
            System.out.println(mapTargetName.get("name"));
        }
    }
}

```

<C/C++>

```

//call I/F with arguments
std::string const    strStartProtocol("Protocol.startProtocol");
std::map<std::string, xmlrpc_c::value> Param;
std::pair<std::string, xmlrpc_c::value> ParamMem1("executionType",
xmlrpc_c::value_string("EXECUTION_TYPE_MANUAL_MAIN"));
Param.insert(ParamMem1);
xmlrpc_c::value_struct const paramSt(Param);
xmlrpc_c::paramList params;
params.add(paramSt);
Proxy.call(strURL, strStartProtocol, params, &Result2);
std::map<std::string, xmlrpc_c::value> mapResult2 = xmlrpc_c::value_struct(Result2);
xmlrpc_c::value_string strResult2 = mapResult2["result"];
if( strResult2.cvalue() == "OK" )
{
    //wait
    xmlrpc_c::value_int nWait=  mapResult2["wait"];
    std::cout << nWait.cvalue() << std::endl;

    //targetName
    xmlrpc_c::value_array TargetNames = mapResult2["targetName"];
    xmlrpc_c::carray v = xmlrpc_c::value_array(TargetNames).vectorValueValue();
    for( int i = 0; i < v.size(); i++ )
    {
        std::map<std::string, xmlrpc_c::value> structData = xmlrpc_c::value_struct(v[i]);
        xmlrpc_c::value_string strName = structData["name"];
        std::cout << strName.cvalue() << std::endl;
    }
}
}

```

<C#>

```

//call I/F with arguments
XmlRpcStruct Input = new XmlRpcStruct();

```

```

Input.Add("executionType", "EXECUTION_TYPE_MANUAL_MAIN");
XmlRpcStruct Result2 = Proxy.startProtocol(Input);
//result
if (((string)Result2["result"]) == "OK")
{
    //wait
    Console.WriteLine(Result2["wait"].ToString());
    //targetName
    Object[] targetNames = (Object[])Result2["targetName"];
    foreach (XmlRpcStruct Name in targetNames)
    {
        Console.WriteLine(Name["name"].ToString());
    }
}
}

```

<MATLAB>

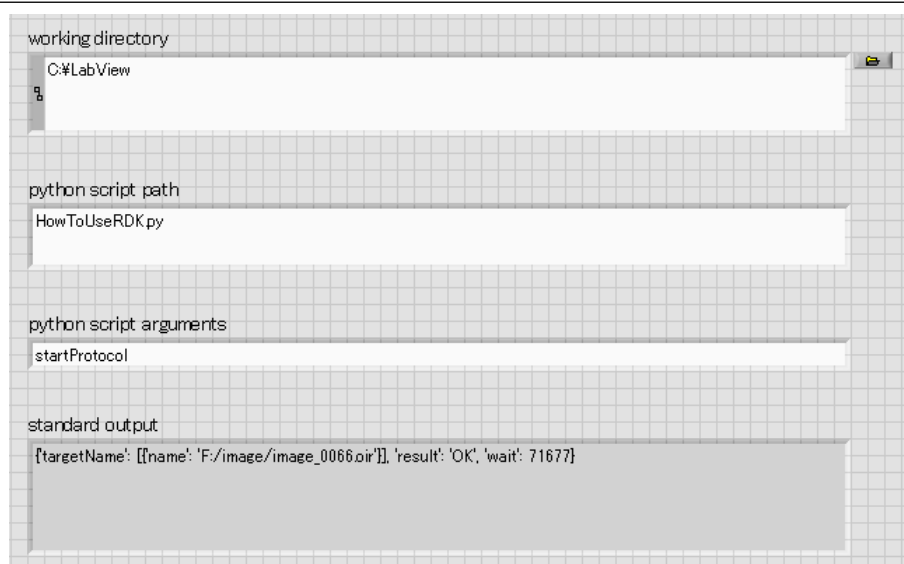
```

%call I/F with arguments
input=java.util.HashMap
input.put('executionType','EXECUTION_TYPE_MANUAL_MAIN')
result2 = proxy.execute('Protocol.startProtocol', input )
% result
if strcmp(result2.get('result'), 'OK')
    % wait
    result2.get('wait')
    % targetName
    targetName = result2.get('targetName')
    for i=1:targetName.size()
        targetName(i)
    end
end
end

```

<LabVIEW>

In case of LabVIEW, the execution results of Python are responded as follows. Since they are strings, the intended values can be extracted by using the string processing function of LabVIEW.



5. Application samples (Explanation of sample codes)

This section introduces the application examples using RDKit together with the explanation of programs. The programming language is explained using “python”.

5.1. Procedures to access to image data

In order to perform the subsequent use cases, it is necessary to access to the image data frequently. This section explains the procedures to access to the image data specifically on the assumption that descriptions in “Image Data” are understood in advance.

5.1.1. Access to File

First of all, access to the file.

```
# open
inputVal = {"filename": filename}
result = proxy.IDA.open(inputVal)
fileId = result["fileId"]
```

5.1.2. Access to Group

Then, access to Group.

Acquire the number of Groups and access to the specified Group.

In the following codes, “groupIndex = 0” is shown, which means that the first Group is accessed.

```
# getNumOfGroups
inputVal = {"fileId": fileId}
result = proxy.IDA.getNumOfGroups(inputVal)

# getGroup
groupIndex = 0
inputVal = {"fileId": fileId,
```

```
"group": groupIndex}
result = proxy.IDA.getGroup(inputVal)
groupId = result['groupId']
```

5.1.3. Access to Level

Then, access to Level.

Acquire the number of Levels.

```
# getNumOfLevel
inputVal = {"gropuld": groupId}
result = proxy.IDA.getNumOfLevel(inputVal)
```

5.1.4. Access to Area

Then, access to Area.

Specify Group and Level described above, and then access to Area.

Acquire the number of Areas first, and access to the specified Area.

In the following codes, “levelIndex = 0” and “areaIndex = 0” are shown, which mean that the first Area of the first Level is accessed.

```
# getNumOfArea
levelIndex= 0
inputVal = {"groupId": groupId,
            "level": levelIndex}
result = proxy.IDA.getNumOfArea(inputVal)

# getArea
areaIndex = 0
inputVal = {"groupId": groupId,
            "level": levelIndex,
            "area": areaIndex}
result = proxy.IDA.getArea(inputVal)
areald = result['areald']
```

5.1.5. Access to Channel

Then, access to Channel.

Specify the Area described above, and acquire ID of the Channel in the image.

In the following example, the list of Channel ID is stored in the variable “channelIdList”.

```
# get channel id list
inputVal = {"areald": areald,
            "key": "EnabledChannelIdList"}
result = proxy.IDA.getProperty(inputVal)
channelIdList = result['value']
```

5.1.6. Access to Lambda/Z/T-axis

This section explains the procedure to access to the dimensions: Lambda-, Z- and T-axis.

At first, acquire the axis of the target Area.

The following shows one of examples to acquire the axis. This example shows that if bExistL is True, the Lambda-axis exists and if bExistL is False, the Lambda-axis does not exist. In the same manner, this example shows whether the Z-axis or T-axis exists or not with respect to bExistZ or bExistT respectively.

```
# get Axes
bExistL = False
bExistZ = False
bExistT = False
inputVal = {"areald": areald,
            "key": "Axes"}
result = proxy.IDA.getProperty(inputVal)
axes = result['value']
for i in axes:
    if i == "TIMELAPSE":
        bExistT = True
    elif i == "LAMBDA":
        bExistL = True
    elif i == "ZSTACK":
        bExistZ = True
```

5.1.7. Access to Image

Then, acquire the specified image from the multi-dimension image composed of Lambda, Z, T and Channel.

The image becomes an object called "Image" and you will acquire that object.

```
# get Image
axisInfo = createAxis(lambdaIndex, zIndex, tIndex,
                      bExistL, bExistZ, bExistT)
inputVal = {"areald": areald,
            "channelId": channelIdList[chIndex],
            "axisInfo": axisInfo}
result = proxy.IDA.getImage(inputVal)
imageId = result['imageId']
```

"createAxis" is the internal function defined separately.

```
def createAxis(LIndex, ZIndex, TIndex, bExistL, bExistZ, bExistT):
    # create axsi info
    if bExistL is True and bExistZ is False and bExistT is False:
        # XYL
        axisInfo = [{"axisName": "LAMBDA",
                     "index": LIndex}]
```



```

elif bExistL is False and bExistZ is True and bExistT is False:
    # XYZ
    axisInfo = [{"axisName": "ZSTACK",
                  "index": ZIndex}]
elif bExistL is False and bExistZ is False and bExistT is True:
    # XYT
    axisInfo = [{"axisName": "TIMELAPSE",
                  "index": TIndex}]
elif bExistL is True and bExistZ is True and bExistT is False:
    # XYLZ
    axisInfo = [{"axisName": "LAMBDA",
                  "index": LIndex},
                 {"axisName": "ZSTACK",
                  "index": ZIndex}]
elif bExistL is False and bExistZ is True and bExistT is True:
    # XYZT
    axisInfo = [{"axisName": "ZSTACK",
                  "index": ZIndex},
                 {"axisName": "TIMELAPSE",
                  "index": TIndex}]
elif bExistL is True and bExistZ is False and bExistT is True:
    # XYLT
    axisInfo = [{"axisName": "LAMBDA",
                  "index": LIndex},
                 {"axisName": "TIMELAPSE",
                  "index": TIndex}]
elif bExistL is True and bExistZ is True and bExistT is True:
    # XYLZT
    axisInfo = [{"axisName": "LAMBDA",
                  "index": LIndex},
                 {"axisName": "ZSTACK",
                  "index": ZIndex},
                 {"axisName": "TIMELAPSE",
                  "index": TIndex}]
else:
    # XY, Line Scan iamge, point scan image, Multipoint image
    axisInfo = [{"axisName": "TIMELAPSE",
                  "index": TIndex}]

return axisInfo

```

5.1.8. Acquiring intensity data

Then, acquire the actual intensity data.

In the following codes, the intensity data is stored in imageData.

Input the image size of the image to be acquired to “width” and “height”.

```
# get Image Body
imageData = getImageBody(proxy, imageId, width, height)
```

“getImageBody” is the internal function and it is implemented as follows.

“while result[‘continue’]”: The following processing acquires the image in several times when the image data is too large to acquire at a time.

```
def getImageBody(proxy, imageId, width, height):
    # get image body
    rect = {"x": 0, "y": 0, "width": width,
            "height": height}
    inputVal = {"imageId": imageId,
                "compressType": "NONE",
                "rect": rect}
    result = proxy.IDA.getImageBody(inputVal)
    image = result['data'].data
    while result['continue']:
        # until all the image data is acquired.
        inputVal = {"imageId": imageId}
        result = proxy.IDA.getNextImageBody(inputVal)
        image = image + result['data'].data
    return image
```

5.1.9. Converting to intensity data array

This section explains the procedures to convert the acquired intensity data to the format easy to access from the programming language. At first, convert the acquired intensity data to the 16bit-width array. Then, convert from the one-dimensional array to the two-dimensional array so that the array can be treated as the image easily.

```
frame = numpy.frombuffer(imageData, dtype=numpy.uint16)
frame = frame.reshape(imageHeight, imageWidth)
```

5.1.10. Acquiring image size

If you want to acquire the image size of the image data, follow the procedures described below.

```
# get Image size
inputVal = {"imageId": imageId}
result = proxy.IDA.getImageSize(inputVal)
width = result['width']
height = result['height']
```

5.1.11. Cleaning up

Now, you can access to the image data using procedures described above. When these accesses are finished, it is necessary to release the acquired `imageId`, `areald`, `groupId` and `fileId`. Release them in the reverse order of the acquisition.

```
# release Image
inputVal = {"imageId": imageId}
result = proxy.IDA.releaseImage(inputVal)

# release area
inputVal = {"areald": areald}
result = proxy.IDA.releaseArea(inputVal)

# release group
inputVal = {"groupId": groupId}
result = proxy.IDA.releaseGroup(inputVal)

# close
inputVal = {"fileId": fileId}
result = proxy.IDA.close(inputVal)
```

5.1.12. Others

As the method to acquire only the intensity data of the specified image easily, the “getImage” function is prepared in the sample program “/SampleProgram/Python/UC_HelperFunction/rdkida.py”. This function executes a series of processings described above (excluding the acquisition of image size) and returns the intensity data. For details, refer to the files described above.

When extracting all the intensity data from the XYZ stack image, it takes time if executing from “open” of the file each time, such as “getImage” in “rdkida.py”. In such a case, since “fileId”, “groupId” and “areald” described above will not change, the speed of this process can be accelerated by retaining them and using repeatedly after they are acquired. In this case, it is recommended to use “class RDKIDA” in “rdkida.py”. Please use them properly according to your intended use.

5.2. Cell Tracking out

5.2.1. Sample program

The sample program described in this section is in

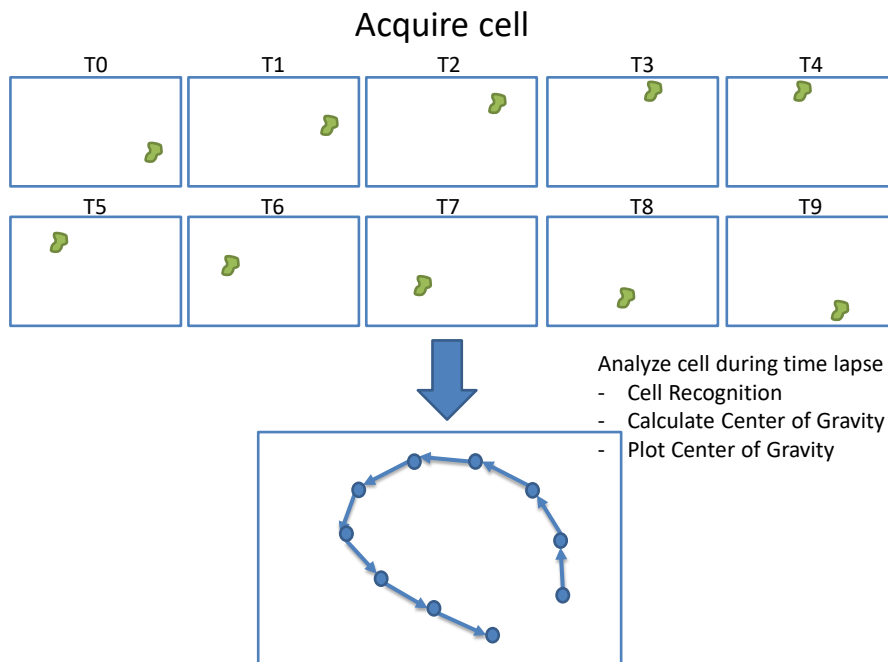
“FV31S-RMTDK/SampleProgram/Python/UC_CellTrakingout/ CellTrakingOut_XYT.py”.

Be sure to set the IPAdress in “FV31S-RMTDK/SampleProgram/Python/UC_HelperFunction/config.ini”.

5.2.2. Contents of use case

It is assumed to track the target which moves during Time-Lapse observation by controlling the motorized XY stage.

This section assumes the example where the target of interest always comes to the center of the screen.



5.2.3. I/F to be used and order

This section explains the overview of the following programs.

- Start imaging.
- Analyze the following center of gravity positions and feedback to the motorized XY stage at a regular interval.
 - Acquire the tracking parameter.
 - Acquire the axis number of Time-Lapse which completes imaging.
 - Calculate the center of gravity position of the observation target with respect to the T-axis frame described above.
 - If the center of gravity position is shifted from the center position by more than a certain value, control the motorized XY stage so that the center of gravity position comes to the center.

Note that the settings required for Time-Lapse observation must have been completed using GUIs in advance.

Start imaging

The start of imaging can be written as follows.

The file path during imaging is stored in "fileName". The acquired image can be analyzed using this file path.

```
inputVal = {"executionType": "EXECUTION_TYPE_MANUAL_MAIN"}
result = proxy.Protocol.startProtocol(inputVal)
targetName = result["targetName"]
fileName = targetName[0]
fileName = fileName['name']
```

Acquire the file name of the image being acquired

If the imaging starts as described above, the file name of the image being acquired can be acquired as its return value.

If the imaging starts using GUI, the file name of the image being acquired can be acquired as follows.

```
# get acquiring file name
result = self.proxy.Protocol.getAcquiringFiles()
fileName = result['targetName']
fileName = fileName[0]
fileName = fileName['name']
```

Acquire various image conditions

To execute subsequent processings, the image size, bit number of one pixel and length are required. These items are acquired in this example.

```
# get iamge size, bit count/pixel, pixel length
anaRDKObj = rdkida.RDKIDA(self.proxy, fileName)
anaRDKObj.openFile()
groupId = anaRDKObj.getGroup(0)
areald = anaRDKObj.getArea(groupId, 0, 0)
imageId = anaRDKObj.getImage(areald, 0, 0, 0, 0)
# get image size
imageWidth, imageHeight = anaRDKObj.getImageSize(imageId)
# get bitCount
result = anaRDKObj.getProperty_ChannelInfo(areald, 0)
bitCount = result['bitCount']
# get pixel length
pixelLength = anaRDKObj.getProperty_PixelLength(areald)
xlengthPerPixel = pixelLength[0] * 1000 # convert from um to nm
ylengthPerPixel = pixelLength[1] * 1000 # convert from um to nm
anaRDKObj.releaseImage(imageId)
```

Analyze the center of gravity position and feed back to the motorized XY stage

Calculate the center of gravity position of the acquired image and feedback to the motorized XY stage.

This program repeats the processings using the “while” sentence in order to execute processings at a constant interval as follows. “Processing” shown below is the processing explained later.

```
while not self.stop_event.is_set():
    Processing
```

Acquire the axis number of Time-Lapse which completes imaging.

Check the progress of imaging.

To do so, use “getProtocolProgress I/F”.

When the “executionType” is “EXECUTION_TYPE_MANUAL_MAIN”, this I/F can acquire the scan state, number of planned times of series and number of completed times as return values.

In this example, the scan state is inputted to the variable “seriesState” and the axis number of T which completes imaging to “analyzedT” respectively.

```
# get acquired time frame.  
inputVal = {"executionType": "EXECUTION_TYPE_MANUAL_MAIN"}  
result = self.proxy.Protocol.getProtocolProgress(inputVal)  
seriesState = result['series']  
analyzedT = seriesState['completeT'] - 1
```

Calculate the center of gravity position

To calculate the center of gravity position, acquire the image data of the axis of T which completes imaging.

```
try:  
    # get image buffer  
    imageId = anaRDKObj.getImage(areaId, 0, 0, analyzedT, 0)  
    frame = anaRDKObj.getImageBody(  
        imageId, imageWidth, imageHeight)  
    anaRDKObj.releaseImage(imageId)
```

Then, convert the acquired image data to the “opencv” format to execute the subsequent “opencv” image processing. Since some “opencv” image processings cannot deal with the 16bit image, they are changed to 8bit in this example.

```
# convert to numpy format  
frame = numpy.frombuffer(frame, dtype=numpy.uint16)  
frame = frame.reshape(imageHeight, imageWidth)  
frame = numpy.uint8(255.0 * frame / (2 ** bitCount))  
colorFrame = cv2.cvtColor(frame, cv2.COLOR_GRAY2RGB)
```

Then, execute some image processings to calculate the center of gravity position.

In this sample, the center of gravity position is calculated by the following algorithms.

(a) Multiplication by constant value:

The visibility is improved by multiplying by the constant value when using the dark sample.

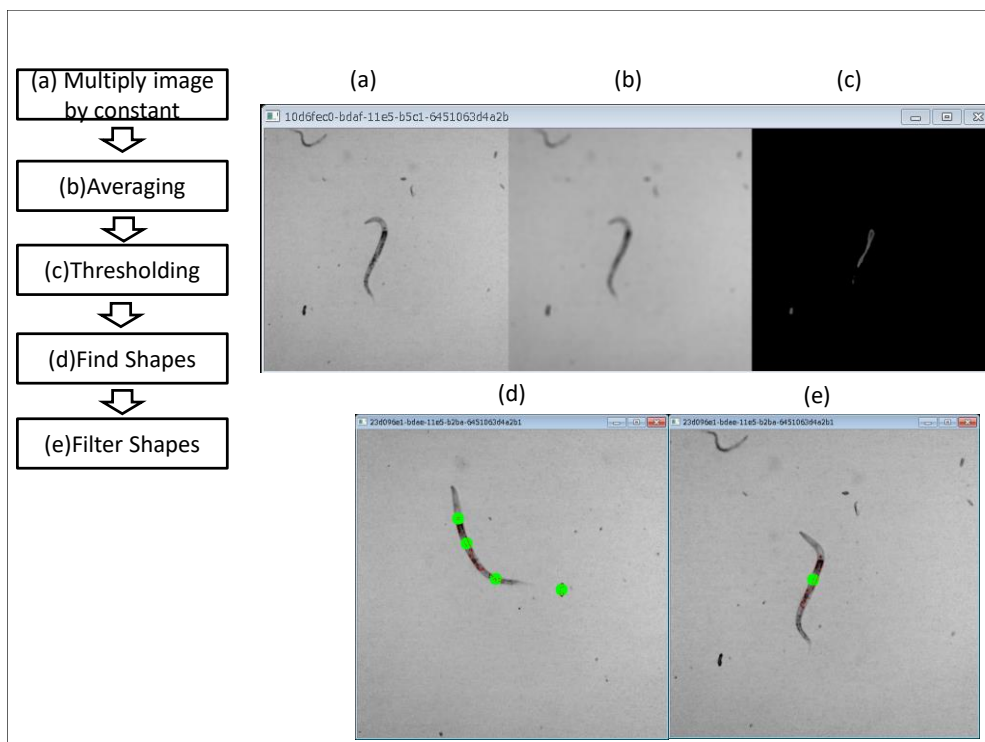
(b) Averaging:

The random noise is eliminated.

(c) Binarization:

(d) Extraction of shapes:

(e) Selection of extracted shapes and calculation of the center of gravity position: In order to narrow down the shapes to one when shapes are extracted, the area of the shape is calculated and the value more than the certain level of area is defined as the shape.



These algorithms are written as follows by “python” codes.

(a) Multiplication by constant value

```
# Multiply
frame_mul = cv2.multiply(frame, multiplyVal)
```

(b) Averaging

```
# Averaging
frame_mul_ave = cv2.blur(frame_mul, (5, 5))
```

(c) Binarization

```
# threshold
if self.threshAlgorithm.get() == 'THRESH_TOZERO':
    ret, frame_mul_ave_thresh = cv2.threshold(frame_mul_ave, threshInt,
                                              255, cv2.THRESH_TOZERO)
else:
    ret, frame_mul_ave_thresh = cv2.threshold(frame_mul_ave, threshInt,
                                              255, cv2.THRESH_TOZERO_INV)
```

(d) Extraction of shapes

```
# Extract shapes
test = frame_mul_ave_thresh + 0
contours, hierarchy = cv2.findContours(test, cv2.RETR_TREE,
                                       cv2.CHAIN_APPROX_SIMPLE)
```

(e) Selection of extracted shapes and calculation of the center of gravity position

“areaVal” is a value specified by the user, and the center of gravity is calculated for the shape larger than that value. The calculated center of gravity position is displayed on the image as a green circle.

```
for cnt in contours:
    # extract a target contour
    # print "area", cv2.contourArea(cnt)
    if cv2.contourArea(cnt) > areaVal:
        print('area', cv2.contourArea(cnt))

    # calculate center of gravity
    M = cv2.moments(cnt)
    if M["m00"] != 0:
        (cx, cy) = (int(M["m10"] / M["m00"]),
                    int(M["m01"] / M["m00"]))
        # display center of gravity
        cv2.circle(
            colorFrame, (cx, cy), 10, (0, 255, 0), -1)
        print('cx', cx, 'cy', cy, 'area', cv2.contourArea(cnt))

    # display contours
    cv2.drawContours(colorFrame, cnt, -1, (0, 0, 255), 1)
```

Feedback to the motorized XY stage

At first, acquire the current position of the XY stage.

```
# get current xy stage position
inputVal = {"settingId": "XY_STAGE_POSITION_SETTING"}
result = self.proxy.Parameter.getParameter(inputVal)
stageX = result["x"]
stageY = result["y"]
```

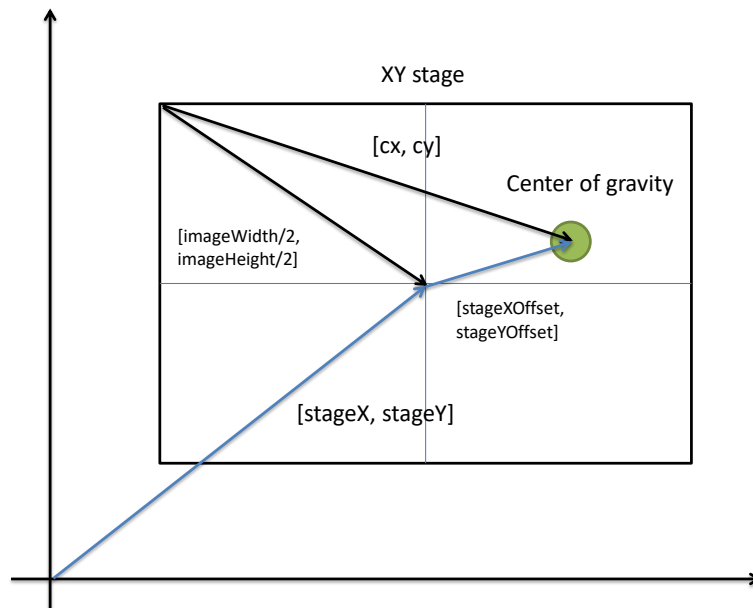
Then, calculate the coordinates of the XY stage where the detected center of gravity position comes to the center of the field of view.

The calculation formulas are shown below, but also refer to the following diagram.

```
# calculate next stage positions
stageXOffset = ((cx - imageWidth / 2) * xlengthPerPixel)
stageYOffset = ((cy - imageHeight / 2) * ylengthPerPixel)
print('oldX', stageX, 'oldY', stageY)
print('Diffx', abs(cx - imageWidth / 2), 'Diffy', abs(cy - imageHeight / 2))
stageX = int(stageX + xstageDirection * stageXOffset)
stageY = int(stageY + ystageDirection * stageYOffset)
```



```
print('newX', stageX, 'newY', stageY)
```



Send the value calculated last to the XY stage.

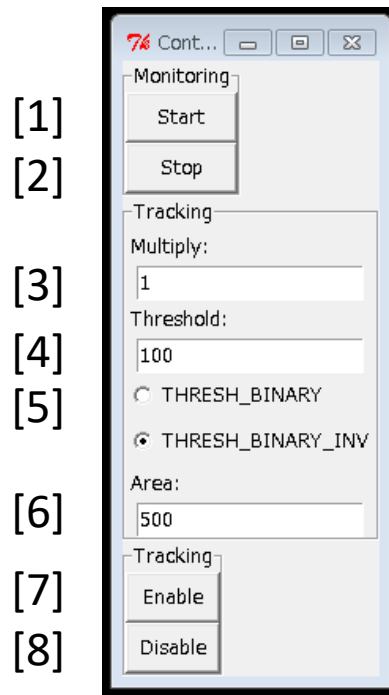
In this example, the feedback is applied if the center of the field of view is away from the center of gravity position by more than 10% of the image size.

```
# feedback
if self.feedback == 1 and ( abs(cx - imageWidth / 2) > imageWidth * 0.1 or
    abs(cy - imageHeight / 2) > imageHeight * 0.1):
    inputVal = {"settingId": "XY_STAGE_POSITION_SETTING",
        "x": stageX,
        "y": stageY,
        "escapeEnabled": False}
    result = self.proxy.Parameter.setParameter(inputVal)
    print('cx, cy', cx, cy)
    print('set', result, inputVal)
    inputVal = {"settingId": "XY_STAGE_POSITION_SETTING"}
    result = self.proxy.Parameter.getParameter(inputVal)
    print('get', result)
    break
```

5.2.4. GUI sample program

This sample code includes following simple GUIs to improve the convenience when executing the sample code. For procedures to create GUIs, see source codes.

This section explains the procedures to operate these GUIs.



- [1] Button to start from acquiring the image data to analyzing the center of gravity position and displaying to the image.
- [2] Button to stop [1].
- [3] Parameter for analyzing the center of gravity position. This parameter multiplies by the value inputted here for the original image.
- [4] Parameter for analyzing the center of gravity position. Threshold of binarization
- [5] Parameter for analyzing the center of gravity position.
THRESH_BINARY: The value less than the threshold is 0 and others are 255.
THRESH_BINARY_INV: The value larger than the threshold is 0 and others are 255.
- [6] Parameter for analyzing the center of gravity position. Threshold when there are multiple detected shapes. Area of the shape. Only the shape which has the area larger than the specified value is extracted.
- [7] Button to feedback the center of gravity position to the stage coordinates.
- [8] Button to release the above.

Summary

This use case introduced the procedures to implement the tracking by feeding back the center of gravity position of the target object to the stage coordinates.

5.3. Automated large FOV MATL set up

5.3.1. Sample program

The sample program described in this section is in

"FV31S-RMTDK/SampleProgram/Python/UC_AutomatedLargeFOVMATLsetup/AutomatedLargeFOVMATLsetup.py".

Be sure to set "IPAddress" to "FV31S-RMTDK/SampleProgram/Python/UC_HelperFunction/config.ini".

Also, set the parameters in the program to

"FV31S-RMTDK/SampleProgram/Python/UC_AutomatedLargeFOVMATLsetup/Parameter.ini".

5.3.2. Contents of use case

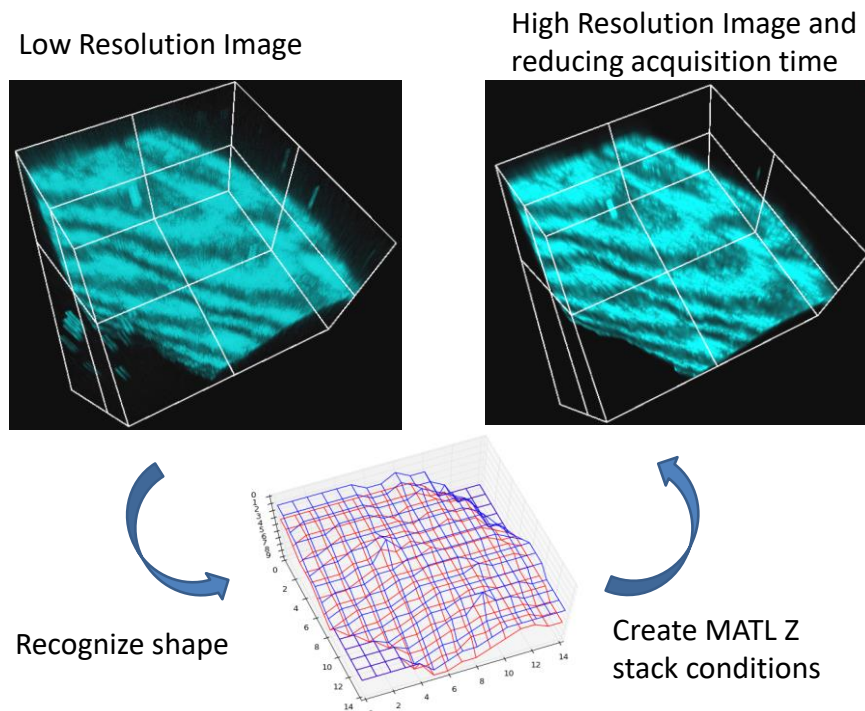
This use case is assumed to complete the ZStack observation over multiple fields of view within a short time.

Assume following steps.

1. Acquire the entire image with low resolution at high-speed
2. Extract the outline of the entire image from the image described above.
3. Change to the high resolution imaging.
4. Create the MATL observation range from the outline described above.
5. MATL imaging

The difference from the ZStack imaging with normal MATL is that the outline is extracted from the entire image acquired with low resolution at high-speed. If the outline can be extracted, the number of ZStack images can be reduced from the area where target objects are not existed to shorten the imaging time, which can be compensated for improving the S/N of imaging area.

The subsequent sections explain 2, 3, and 4 described above.



5.3.3. I/F to be used and order

The entire flow to implement the use case described above is described as follows.

The subsequent sections explain the procedures to use I/F by each step.

Extract the outline from the entire image

In this example, acquire the XYZ image in advance as an entire image and extract the outline of this image. Acquire the image size and the ZStack information (start position, end position and step size).

At first, open the image file.

```
# open file
fileName = analyzedFile
anaRDKObj = rdkida.RDKIDA(proxy, fileName)
anaRDKObj.openFile()
groupId = anaRDKObj.getGroup(0)
areald = anaRDKObj.getArea(groupId, 0, 0)
imageId = anaRDKObj.getImage(areald, 0, 0, 0, 0)
```

Then, acquire the image size.

```
# get image size
imageWidth, imageHeight = anaRDKObj.getImageSize(imageId)
```

Then, acquire the ZStack information.

```
# get start, end , step value
zStart, zEnd, zStep, numOfSlice = anaRDKObj.getAxisProperty(
    areald, "ZSTACK")
# get current zBase position
inputVal = {"settingId": "LSM_Z_COORDINATE_BASE_SETTING",
            }
result = proxy.Parameter.getParameter(inputVal)
```

Then, convert the acquired ZStack value from the absolute value to the relative value.

```
# convert form absolute value to relative value
zStart = int(zStart * 1000) # convert from um to nm
zEnd = int(zEnd * 1000) # convert from um to nm
zStep = int(zStep * 1000) # convert from um to nm
zBase = result['originPosition']
zStart = zStart - zBase
zEnd = zEnd - zBase
```

Then, close the file.

```
# close file
anaRDKObj.releaseImage(imageId)
anaRDKObj.releaseArea(areald)
anaRDKObj.releaseGroup(groupId)
anaRDKObj.closeFile()
```

Then, calculate the direction in which ZStack was acquired. This flag is used for creating the subsequent observation conditions.

```
if zStart < zEnd:
    zCoordinateDirection = 1
else:
    zCoordinateDirection = -1
print("zCoordinateDirection", zCoordinateDirection)
```

Then, load all images of Z stack and judge whether they are larger than the threshold or not.

In particular, divide X and Y directions of the Stitching image specified in "fileName" to "dividex" and "dividey". Calculate the intensity value of the Z slice per divided area, and judge whether that value is larger than the threshold or not. If the value is larger than the threshold, save 1 to "threshResult", and if not, save 0.

```
# load all frames.
threshResult = numpy.zeros([numOfSlice, dividex, dividey])
for sliceIndex in range(0, numOfSlice):
    print("progress[%]", int((float(sliceIndex) / float(numOfSlice)) * 100.0))
    frame = rdkida.getImage(proxy, fileName, 0, 0, 0, sliceIndex, 0, 0)
    frame = numpy.frombuffer(frame, dtype=numpy.uint16)
    frame = frame.reshape(imageWidth, imageHeight)
    import cv2
    # apply average filter
    image = cv2.blur(frame, (40, 40))
    # Judge wheter average value is greater than threshold.
    for i in range(0, dividey):
        for j in range(0, dividex):
            xStart = (int)(j * imageHeight / dividex)
            xEnd = (int)((j + 1) * imageHeight / dividex)
            yStart = (int)(i * imageWidth / dividey)
            yEnd = (int)((i + 1) * imageWidth / dividey)
            value = numpy.average(frame[yStart: yEnd, xStart: xEnd])
            # print sliceIndex, i, j, value
            if threshold <= value:
                threshResult[sliceIndex, j, i] = 1
            continue
    print("progress[%]", "100")
```

Then, from the "threshResult" described above, calculate the slice number where 1 appears at first and the slice number where 1 appears at last per divided area, and store them in the variable "zStackRange".

```
# detect the start and end positions of a sample.
zStackRange = [] # x,y,start,end
```

```

for i in range(0, dividey):
    for j in range(0, dividex):
        zStartIndex = (int)(numOfSlice / 2)
        zEndIndex = (int)(numOfSlice / 2)
        for zIndex in range(0, numOfSlice):
            if threshResult[zIndex, j, i] == 1:
                zStartIndex = zIndex
                break
        for zIndex in range(0, numOfSlice)[::-1]:
            if threshResult[zIndex, j, i] == 1:
                zEndIndex = zIndex
                break
        zStackRange.append([j, i, zStartIndex, zEndIndex])
# print zStackRange

```

Change to high resolution imaging

Change to settings for high resolution imaging.

If there is an image acquired under a certain condition, there is I/F which changes the imaging parameters to same conditions as those of that image. This I/F is executed in this example.

I/F to be used is "loadParameter I/F".

```

# load scanning parameter
inputVal = {"targetName": scanParameterPath
            }
result = proxy.Parameter.loadParameter(inputVal)

```

Create MATL observation range from outline

Register the MATL ROI from the information of the extracted outline.

At first, create "Define Matrix" of the size of (dividex) x (dividey), and then, change the ZStack observation range per area from the outline information.

"Define Matrix" can be created by the following programs.

I/F to be used are "setParameter" and "0MATL_ROI_CREATE".

```

# Create define matrix
inputVal = {"settingId": "MATL_ROI_CREATE",
            "acquisitionType": "LSM_IMAGING",
            "roiType": "DEFINE_MATRIX",
            "column": dividex,
            "row": dividey}
Result = proxy.Parameter.setParameter(inputVal)

```

Then, change the ZStack observation conditions of each area in MATL ROI registered.

The overall program is shown below.

```
# Adjust Z parameters
# get registerd list
inputVal = {"settingId": "MATL_ROI_GET_REGISTERD_LIST"}
result = proxy.Parameter.getParameter(inputVal)
registeredList = result['groupList']
registeredList = registeredList[0]
index = 0
totalNumOfSlice = 0
for i in range(0, dividey):
    for j in range(0, dividex):
        zParameter = zStackRange[index]
        zNewStart = zStart + zCoordinateDirection * zParameter[2] * zStep
        zNewEnd = zStart + zCoordinateDirection * zParameter[3] * zStep
        if zNewStart == zNewEnd and i == 0 and j == 0:
            # cannot delete first area. so set minimum zstack parameter
            zNewEnd = zNewEnd + zCoordinateDirection * zStep
        if zNewStart == zNewEnd:
            # delete this area
            inputVal = {"settingId": "MATL_ROI_AREA_DELETE_SETTING",
                        "groupId": registeredList['groupId'],
                        "column": j,
                        "row": i
                       }
            result = proxy.Parameter.setParameter(inputVal)
            print(result, "delete area")
        else:
            print("reset parameter")
            if zNewStart > zNewEnd:
                # set end position
                inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                            "endPosition": zEnd
                           }
                result = proxy.Parameter.setParameter(inputVal)
                print(result, 'zEnd', zEnd)
                # set start position
                inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                            "startPosition": zStart
                           }
                result = proxy.Parameter.setParameter(inputVal)
                print(result, 'zStart', zStart)
```

```

else:
    # set start position
    inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                "startPosition": zStart
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStart', zStart)
    # set end position
    inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                "endPosition": zEnd
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zEnd', zEnd)
    # set step position
    inputVal = {"settingId": "LSM_Z_COORDINATE_STEP_SIZE_SETTING",
                "stepSize": zStep
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStep', zStep)

print "set new parameter"
if zNewStart > zNewEnd:
    print("zNewStart > zNewEnd")
    # set end position
    inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                "endPosition": zNewEnd
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zEnd', zNewEnd)
    # set start position
    inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                "startPosition": zNewStart
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStart', zNewStart)
else:
    print("zNewStart <= zNewEnd")
    # set start position
    inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                "startPosition": zNewStart
                }

```



```

        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zStart', zNewStart)
        # set end position
        inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                    "endPosition": zNewEnd
                    }
        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zEnd', zNewEnd)

    # apply z condition to MATL area
    inputVal = {"settingId": "MATL_ROI_AREA_Z_RANGE_SETTING",
                "groupId": registeredList['groupId'],
                "column": j,
                "row": i
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, "range setting")
    totalNumOfSlice = totalNumOfSlice + 1
    abs(zNewEnd - zNewStart) / zStep
    index = index + 1

```

As the program is very long, it is explained by dividing into a few parts.

The following program acquires the information for accessing to each area of registered “DefineMatrix”.

To do so, use “getParameter” and “MATL_ROI_GET_REGISTERD_LIST” to acquire the group information.

```

# get registerd list
inputVal = {"settingId": "MATL_ROI_GET_REGISTERD_LIST"}
result = proxy.Parameter.getParameter(inputVal)
registeredList = result['groupList']
registeredList = registeredList[0]

```

Next, the following program calculates the start position (zNewStart) and the end position (zNewEnd) of ZStack per each area from the shape information described above.

```

index = 0
totalNumOfSlice = 0
for i in range(0, dividey):
    for j in range(0, dividex):
        zParameter = zStackRange[index]
        zNewStart = zStart + zCoordinateDirection * zParameter[2] * zStep
        zNewEnd = zStart + zCoordinateDirection * zParameter[3] * zStep

```

Next, register the start position and the end position per each area.

As there are conditional branches, they will be explained one by one.

The following condition shows that the start position is same as the end position and the area is a first area. The area where the start position is same as the end position means that it is not necessary to get ZStack and such area must be deleted from the imaging. However, since the first area cannot be deleted due to restrictions of RDK, two ZStack are set as the minimum ZStack.

```
if zNewStart == zNewEnd and i == 0 and j == 0:
    # cannot delete first area. so set minimum zstack parameter
    zNewEnd = zNewEnd + zCoordinateDirection * zStep
```

The following condition shows that the start position is same as the end position but the area is not a first area. In this case, it is not necessary to acquire the image as the above case. Therefore, "MATL_ROI_AREA_DELETE_SETTING" is used to delete such area.

```
if zNewStart == zNewEnd:
    # delete this area
    inputVal = {"settingId": "MATL_ROI_AREA_DELETE_SETTING",
               "groupId": registeredList['groupId'],
               "column": j,
               "row": i
              }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, "delete area")
```

Then, set the original conditions again before setting the new Z condition per each area. This processing avoids the unexpected error which may occur when setting the different conditions per each area.

To change the ZStack condition per each area, use "setParameter" and "MATL_ROI_AREA_Z_RANGE_SETTING". ZStack parameters must be set to this I/F in advance.

Use followings for ZStack parameters:

LSM_Z_COORDINATE_SLICE_NUM_SETTING

LSM_Z_COORDINATE_STEP_SIZE_SETTING

LSM_Z_COORDINATE_START_SETTING

LSM_Z_COORDINATE_END_SETTING

Note that it is not necessary to set all of them. The following program sets only the start position, end position and step number. (Those which are not set will be calculated automatically.)

```
else:
    print("reset parameter")
    if zNewStart > zNewEnd:
        # set end position
        inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                   "endPosition": zEnd
                  }
```

```

        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zEnd', zEnd)
        # set start position
        inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                    "startPosition": zStart
                    }
        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zStart', zStart)
    else:
        # set start position
        inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                    "startPosition": zStart
                    }
        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zStart', zStart)
        # set end position
        inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                    "endPosition": zEnd
                    }
        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zEnd', zEnd)
    # set step position
    inputVal = {"settingId": "LSM_Z_COORDINATE_STEP_SIZE_SETTING",
                "stepSize": zStep
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStep', zStep)

    print("set new parameter")
    if zNewStart > zNewEnd:
        print("zNewStart > zNewEnd")
        # set end position
        inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                    "endPosition": zNewEnd
                    }
        result = proxy.Parameter.setParameter(inputVal)
        print(result, 'zEnd', zNewEnd)
        # set start position
        inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                    "startPosition": zNewStart
                    }

```

```
result = proxy.Parameter.setParameter(inputVal)
print(result, 'zStart', zNewStart)
```

Then, set the Z stack conditions calculated per each area.

```
print("set new parameter")
if zNewStart > zNewEnd:
    print("zNewStart > zNewEnd")
    # set end position
    inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                "endPosition": zNewEnd
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zEnd', zNewEnd)
    # set start position
    inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                "startPosition": zNewStart
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStart', zNewStart)
else:
    print("zNewStart <= zNewEnd")
    # set start position
    inputVal = {"settingId": "LSM_Z_COORDINATE_START_SETTING",
                "startPosition": zNewStart
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zStart', zNewStart)
    # set end position
    inputVal = {"settingId": "LSM_Z_COORDINATE_END_SETTING",
                "endPosition": zNewEnd
                }
    result = proxy.Parameter.setParameter(inputVal)
    print(result, 'zEnd', zNewEnd)
```

At the end, execute the processing to reflect the conditions specified as above to the area.

```
# apply z condition to MATL area
inputVal = {"settingId": "MATL_ROI_AREA_Z_RANGE_SETTING",
            "groupId": registeredList['groupId'],
            "column": j,
            "row": i
            }
```

```
result = proxy.Parameter.setParameter(inputVal)
print(result, "range setting")
```

Now, the shape can be extracted from the low resolution image and it can be registered as ROI of MATL. After this, start the MATL observation using the following I/F.

```
inputVal = {"executionType": " EXECUTION_TYPE_MATL"}
result = proxy.Protocol.startProtocol(inputVal)
```

To stitch the acquired image data, follow the program shown below.

```
# get groupId
fileName = stitchFile
omp2Info = getGroupIdFromOMP2INFO(fileName)
groupId = (omp2Info[0])[0]

# get channel Id
RDKObj = rdkida.RDKIDA(proxy, fileName)
RDKObj.openFile()
groupId2 = RDKObj.getGroup(0)
areaId = RDKObj.getArea(groupId2, 0, 0)
channelIdList = RDKObj.getProperty_ChannelIdList(areaId)
RDKObj.releaseArea(areaId)
RDKObj.releaseGroup(groupId2)
RDKObj.closeFile()

# execute stitching
stitchParam = {"stitchType": "STITCH_DEPTH_WISE",
               "matchingType": "NONE",
               "smoothingType": "NONE",
               "zMatchIndex": 0,
               "tMatchIndex": 0,
               "lMatchIndex": 0,
               "scale": 100,
               "matchingChannelIds":
               channelIdList
               }

inputVal = {"processingId": "MATL_STITCH",
            "targetName": fileName,
            "groupId": groupId,
            "stitchSetting": stitchParam
            }
```

```
result = proxy.Processing.startProcessing(inputVal)
print(result)

while(1):
    inputVal = {"processingId": "MATL_STITCH",
               "targetName": fileName,
               "groupId": groupId}
    result = proxy.Processing.getProcessingProgress(inputVal)
    progress = result['progress']
    print(result)
    if progress['complete'] == 100:
        break
print("finish")
```

Summary

This use case explains the procedures to extract the shape from the ZStack image and feedback those to the MATL ROI conditions. Setting these observation conditions enables to avoid imaging the unnecessary areas and to reduce the imaging time.

5.4. Live Streaming

5.4.1. Contents of use case

In use cases described above, the image saved on HDD is acquired when acquiring the image. To acquire the image data at higher speed, there is a method to acquire the Live image data.

This use case explains that method.

5.4.2. I/F to be used and order

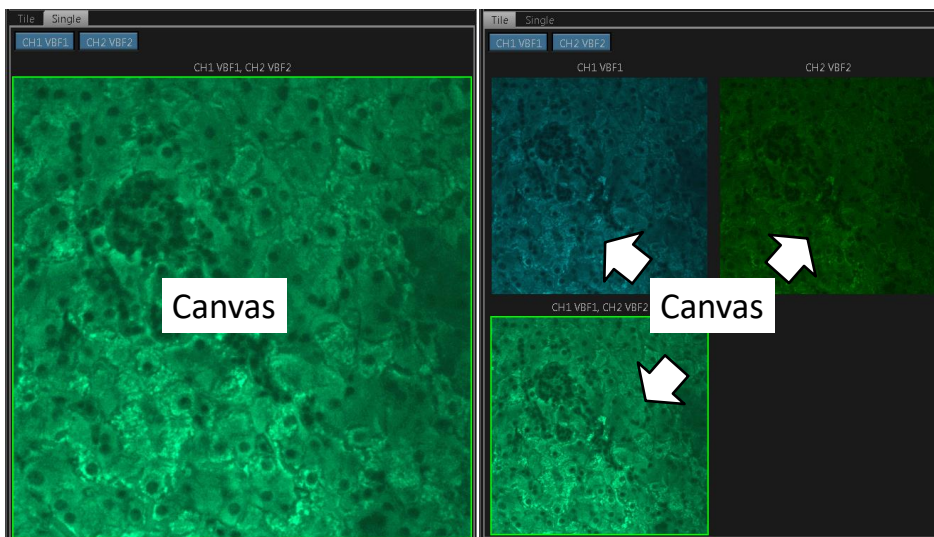
Follow the steps described below to acquire the Live image data.

1. Acquire the Canvas ID.
2. Acquire the list of file formats.
3. Acquire the list of compression types.
4. Acquire the image data from the specified Canvas.

Acquire the Canvas ID

“Canvas” is a range which displays the Live image as shown below. The Canvas ID is a unique ID which indicates the Canvas.

One Canvas ID is displayed in Single view (left picture) and the “number of channels + number of merged images” of Canvas IDs are displayed in Tile view (right picture).



Use the following I/F to acquire the Canvas ID.

```
# get canvas id list
result = proxy.Canvas.getCanvasIdList()
idList = result['idList']
print(result)
```

Acquire the list of file formats

When acquiring the Canvas image, its format can be selected. The selectable formats can be acquired from followings.

```
# getSupportedImageFileFormat
result = proxy.Canvas.getSupportedImageFileFormat()
formatList = result['formatList']
print(result)
```

Acquire the list of compression types

The compression type of the image can be selected. The selectable types can be acquired from followings.

```
# getSupportedCompressType
result = proxy.Canvas.getSupportedCompressType()
compressionTypeList = result['compressionTypeList']
print(result)
```

Acquire the image data from the specified Canvas

Use “getCanvasImage I/F” to acquire the image data of the Canvas.

The following program acquires all the image data as the color image in BMP format without compression, and then the image data of the first Canvas is displayed.

Note that “sessionId” can be any value if determined uniquely. The following program creates “Id” using the

“uuid” function which can acquire the unique value.

```
# getCanvasImage
sessionId = str(uuid.uuid1())
inputVal = {
    "sessionId": sessionId,
    "canvasIdList": idList,
    "format": "BMP",
    "compress": "NONE",
    "width": imageWidth,
    "height": imageHeight,
    "colorType": "COLOR",
    "bitDepth": 8}
result = proxy.Canvas.getCanvasImage(inputVal)
result = result['image']
result = result[0]
image = result['data'].data
while result['continue']:
    result = proxy.Canvas.getCanvasImage(inputVal)
    result = result['image']
    result = result[0]
    image = image + result['data'].data
```

Remarks: The image displayed on Canvas is acquired as “Canvas Image”. If “COLOR” is set to “colorType” as shown above, LUT applies and the value is converted to 8bit. Note that the row data is different from the intensity value. “GRAYSCALE” can also be set to “colorType”, but in this case, LUT is not applied and the intensity data is converted to 8bit.

Summary

This use case explains the procedures to acquire the Live image. As the Live image can be acquired at high-speed, this is useful with the application which requires high-speed.

6. Troubleshooting

- If a large amount of commands of RDK are issued in a short time, following error may occur in a rare case. In this case, you may be able to avoid this phenomenon by reducing the frequency of issuing the commands.

< Error in Pytyon >

```
python socket error: [Errno 10054]
```

- FLUOVIEW software uses port number 8080. If other software has already used 8080, the connection will be failed. In that case, the port number should be changed by doing the following setting.

-
1. Open "C:\Program Files\OlympusMicro\FV30S-SW\eclipse\FV31S-SW.ini" with a texteditor.
 2. Add the following text to the beginning of the file :
-rdkport
<new port number>

Example:

-rdkport
9090

7. Trademark

- MATLAB is the registered trademark of The MathWorks, Inc.
- LabVIEW is the trademark of National Instruments Co.
- Java is the registered trademark of Oracle Corporation, its subsidiaries and affiliated companies in America and other countries.